Securing Cloud Communication

Dr Mukti Padhya

Assistant Professor

NFSU, Gandhinagar

What is Cryptography?

- Cryptography, or cryptology, is the practice and study of techniques for secure communication in the presence of adversarial behavior.

OSI Security Architecture

- The OSI (Open Systems Interconnection) security architecture focuses on Security Attacks, Mechanisms, and Services.
- **Security Attack:** Any action that compromises the security of information owned by an organization.
- **Security Mechanism:** A process that is designed to detect, prevent, or recover from a security attack.
- **Security Service:** A communication service that enhances the security of the data processing systems and the information transfers of an organization.

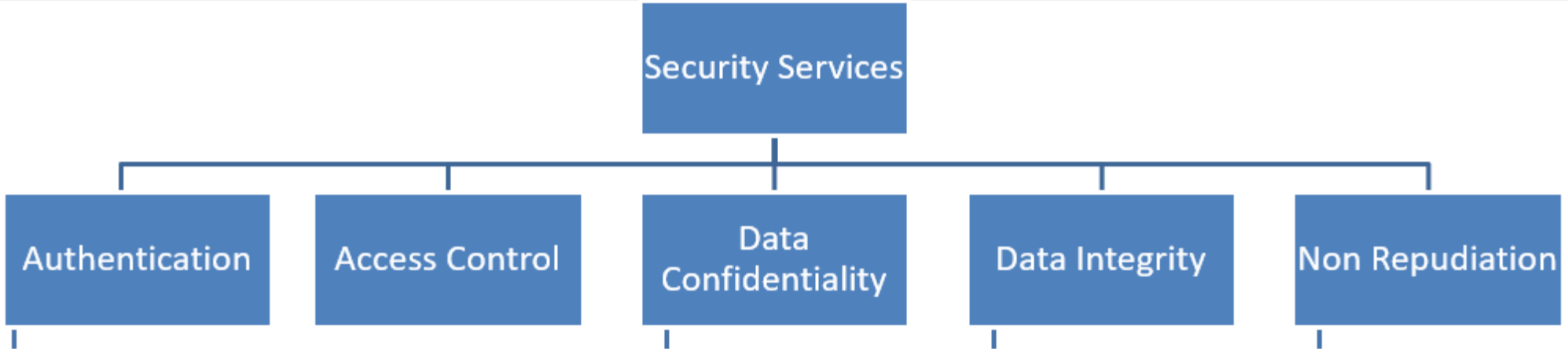
Why we need Cryptography ?

Security Attacks

- A **passive attack** attempts to learn or make use of information from the system but does not affect system resources.
  1. Release of message contents
  2. Traffic analysis
- An **active attack** attempts to alter system resources or affect their operation.
  1. Masquerade
  2. Replay
  3. Modification of messages
  4. Denial of service.

Security Services (X.800)

- X.800 standard defines a security service as a service that is provided by a protocol layer of communicating open systems and that ensures security of the systems or of data transfers.



Security Mechanism (Specific Security)

- **Encipherment:** Hiding or covering data using mathematical algorithms.
- **Digital Signature:** The sender can electronically sign the data and the receiver can electronically verify the signature.
- **Access Control:** A variety of mechanisms that enforce access rights to resources.
- **Data Integrity:** A variety of mechanisms used to assure the integrity of a data unit or stream of data units.
- **Authentication Exchange:** Two entities exchange some messages to prove their identity to each other.

Symmetric Cipher Model (Conventional Encryption)

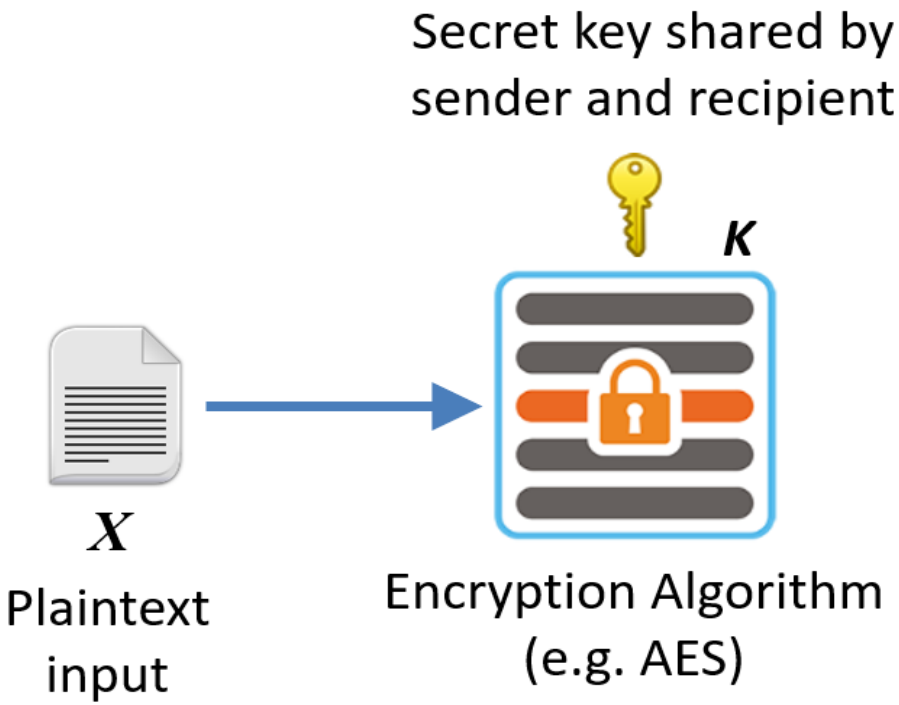


X

Plaintext input

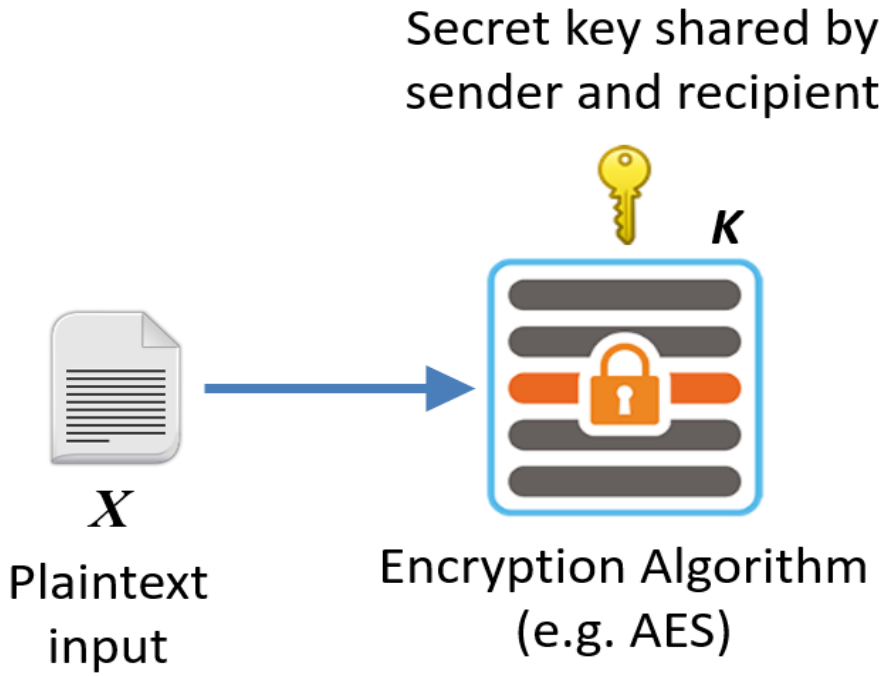
- An original message is known as the **plaintext**, while the coded message is called the **ciphertext**.
- The process of converting from plaintext to ciphertext is known as **enciphering** or **encryption**; restoring the plaintext from the ciphertext is **deciphering** or **decryption**.

Symmetric Cipher Model (Conventional Encryption)



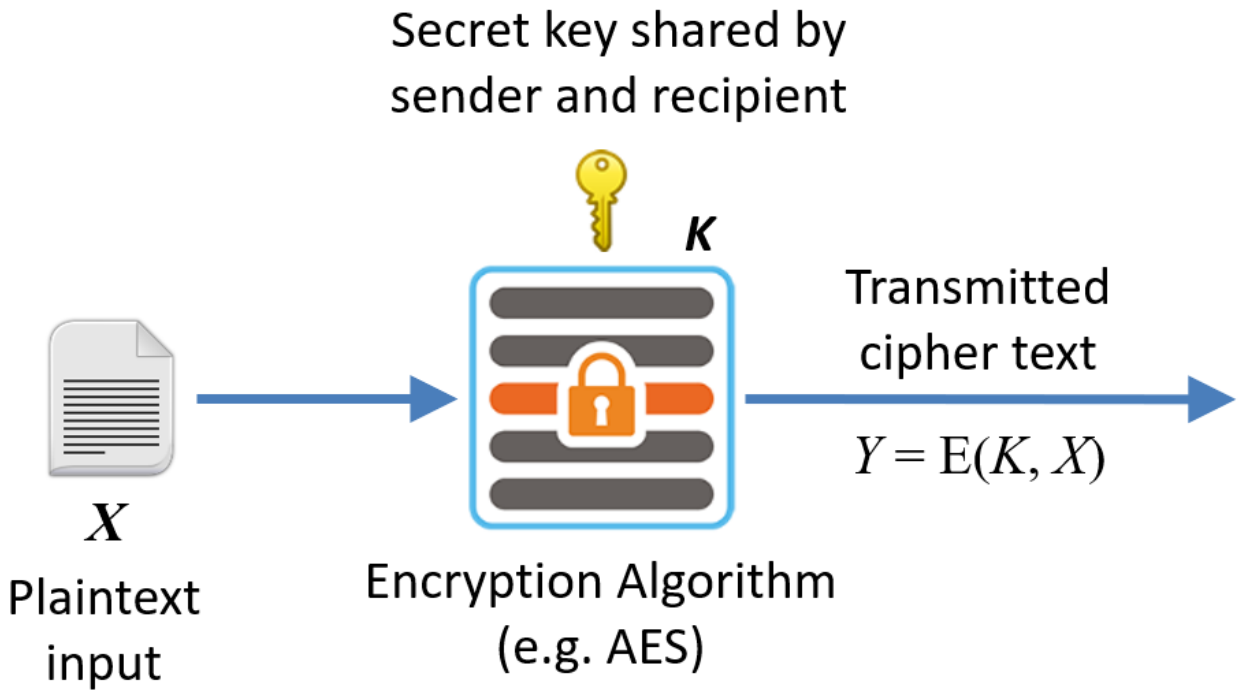
- **Plaintext** is the original intelligible message or data that is fed into the algorithm as input.
- **Encryption algorithm** performs various substitutions and transformations on the plaintext.

Symmetric Cipher Model (Conventional Encryption)



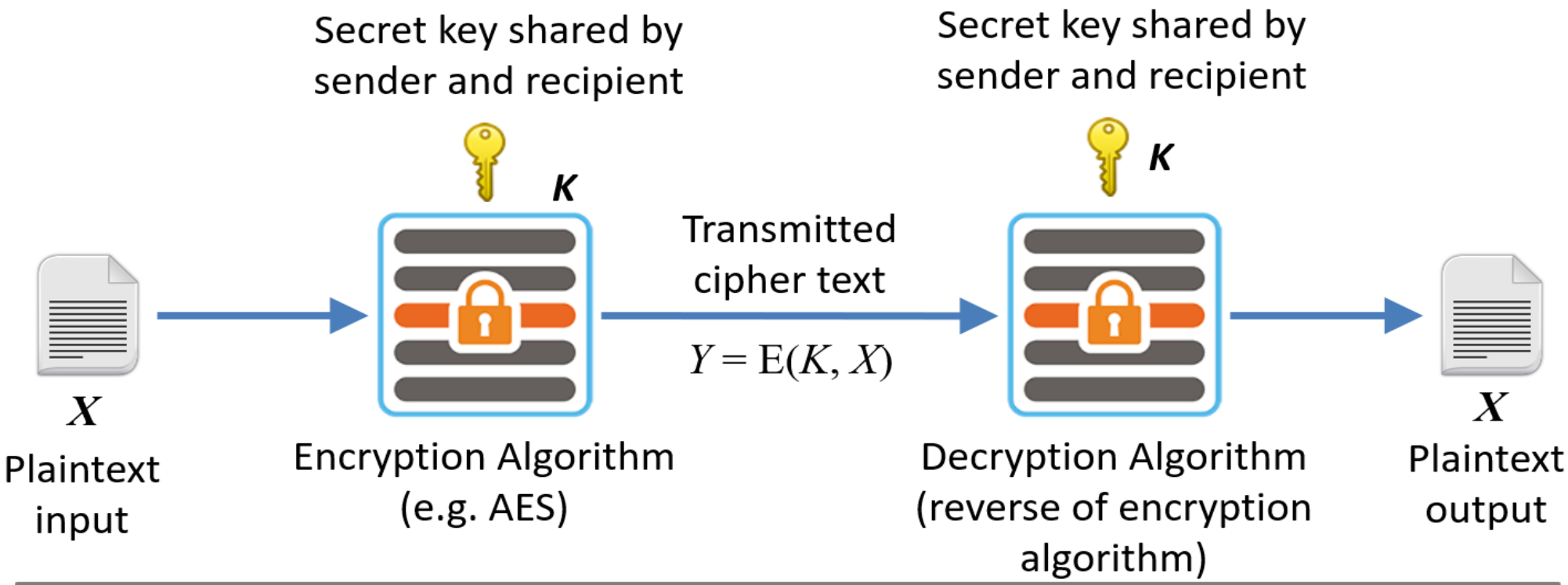
- The **secret key** is also input to the encryption algorithm.
- The key is a value independent of the plaintext and of the algorithm.
- The algorithm will produce a different output depending on the specific key being used at the time.

Symmetric Cipher Model (Conventional Encryption)



- **Ciphertext** is the scrambled message produced as output.
- It depends on the plaintext and the secret key.
- The ciphertext is an apparently random stream of data and, as it stands, is unintelligible.

Symmetric Cipher Model (Conventional Encryption)



- **Decryption algorithm** is essentially the encryption algorithm run in reverse.
- It takes the ciphertext and the secret key and produces the original plaintext.

Types of Encryption Algorithms

- can characterize Encryption system by:
  - ❖ type of encryption operations used
    - ➢ Substitution (each element of PT is mapped into another element)
    - ➢ Transposition (elements in PT are rearranged)
    - ➢ Product (combination of both)
  - ❖ number of keys used
    - ➢ single-key or private or symmetric
    - ➢ two-key or public or asymmetric
  - ❖ way in which plaintext is processed
    - ➢ block
    - ➢ stream

Types of Encryption Algorithms (cotd.)

- type of encryption operations used
  - Substitution (each element of PT is mapped into another element)

```
plain:  a b c d e f g h i j k l m n o p q r s t u v w x y z
cipher: y n l k x b s h m i w d p j r o q v f e a u g t z c
```

  - Transposition (elements in PT are rearranged)

For example, to send the message **"Meet me at the park"** to Bob,

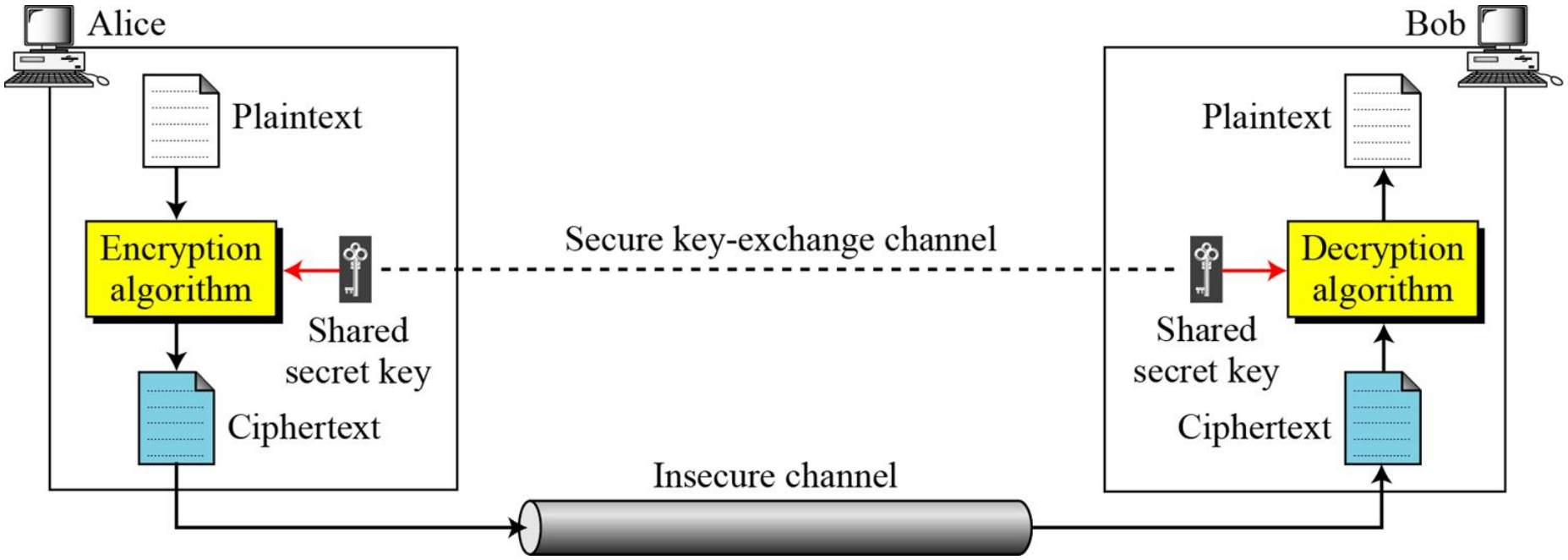
Alice writes



She then creates the ciphertext "MEMATEAKETETHPR".

Types of Encryption Algorithms (cotd.)

- No. of Keys used: Symmetric-key cipher



Symmetric-key cipher

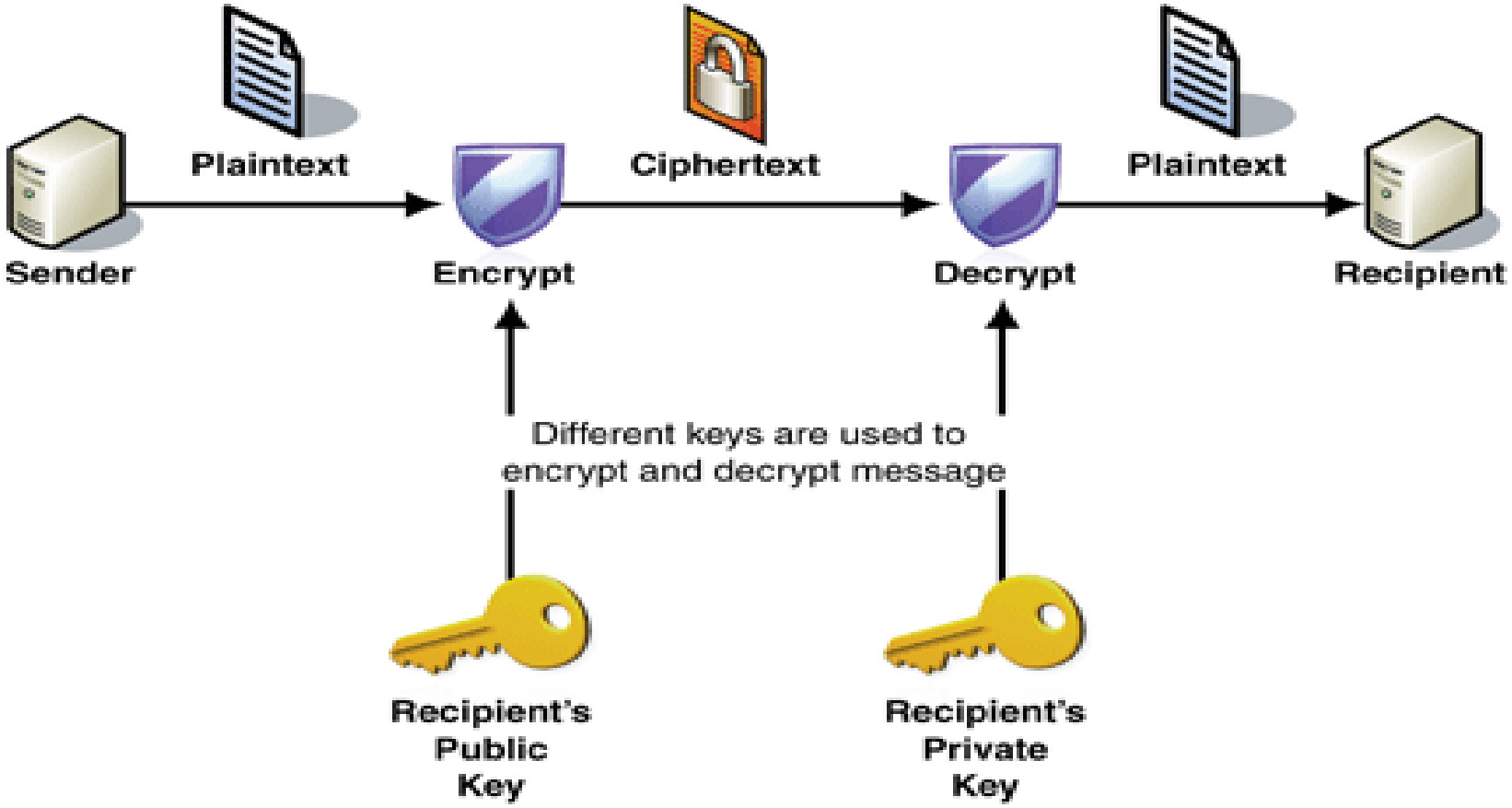
If P is the plaintext, C is the ciphertext, and K is the key,

$$\text{Encryption: } C = E_k(P) \qquad \text{Decryption: } P = D_k(C)$$

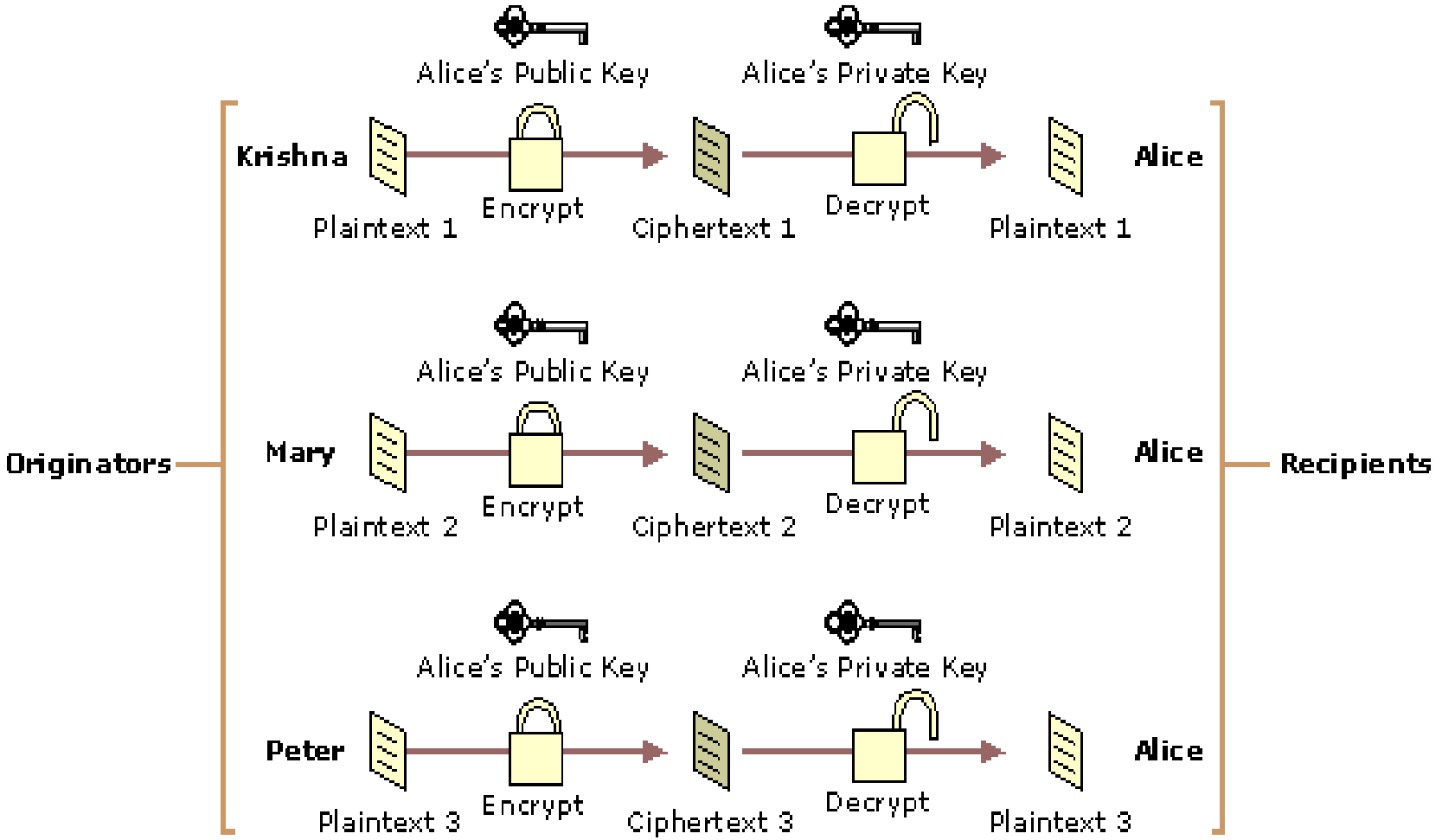
$$\text{In which, } D_k(E_k(x)) = E_k(D_k(x)) = x$$

Types of Encryption Algorithms (cotd.)

- No. of Keys Used: Asymmetric-key Cipher



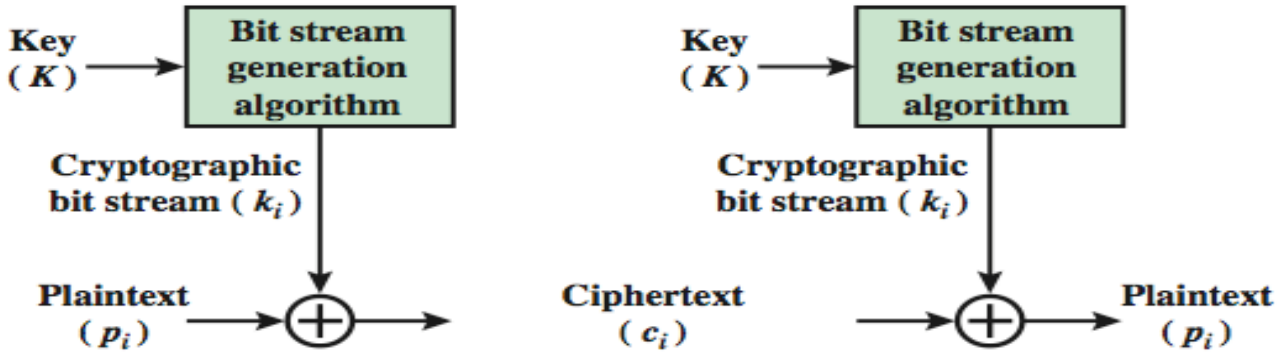
Asymmetric-key Cipher



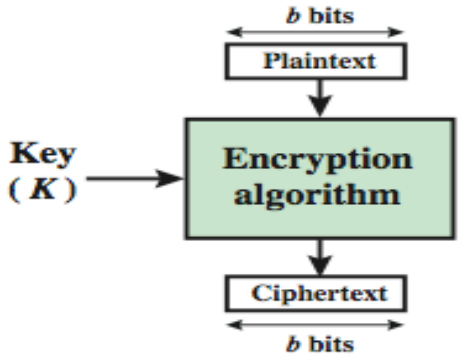
On the basis of	Symmetric Encryption	Asymmetric Encryption
Keys used	It uses a single shared key (secret key) to encrypt and decrypt the message.	It uses two different keys for encryption and decryption.
Size	The size of ciphertext in symmetric encryption could be the same or smaller than the plain text.	The size of ciphertext in asymmetric encryption could be the same or larger than the plain text.
Efficiency	It is efficient as this technique is recommended for large amounts of text.	It is inefficient as this technique is used only for short messages.
Speed	The encryption process of symmetric encryption is faster as it uses a single key for encryption and decryption.	The encryption process in asymmetric encryption is slower as it uses two different keys; both keys are related to each other through the complicated mathematical process.
Purpose	Symmetric encryption is mainly used to transmit bulk data.	It is mainly used in smaller transactions. It is used for establishing a secure connection channel before transferring the actual data.
Security	It is less secured as there is a use of a single key for encryption.	It is safer as there are two keys used for encryption and decryption.
Algorithms	The algorithms used in symmetric encryption are 3DES, AES, DES, and RC4.	RSA, DSA, Diffie-Hellman, ECC, Gamal, and EI.
Existence	It is an old technique.	It is a new technique.

Types of Encryption Algorithms (cotd.)

- way in which plaintext is processed: Block Vs Stream Ciphers

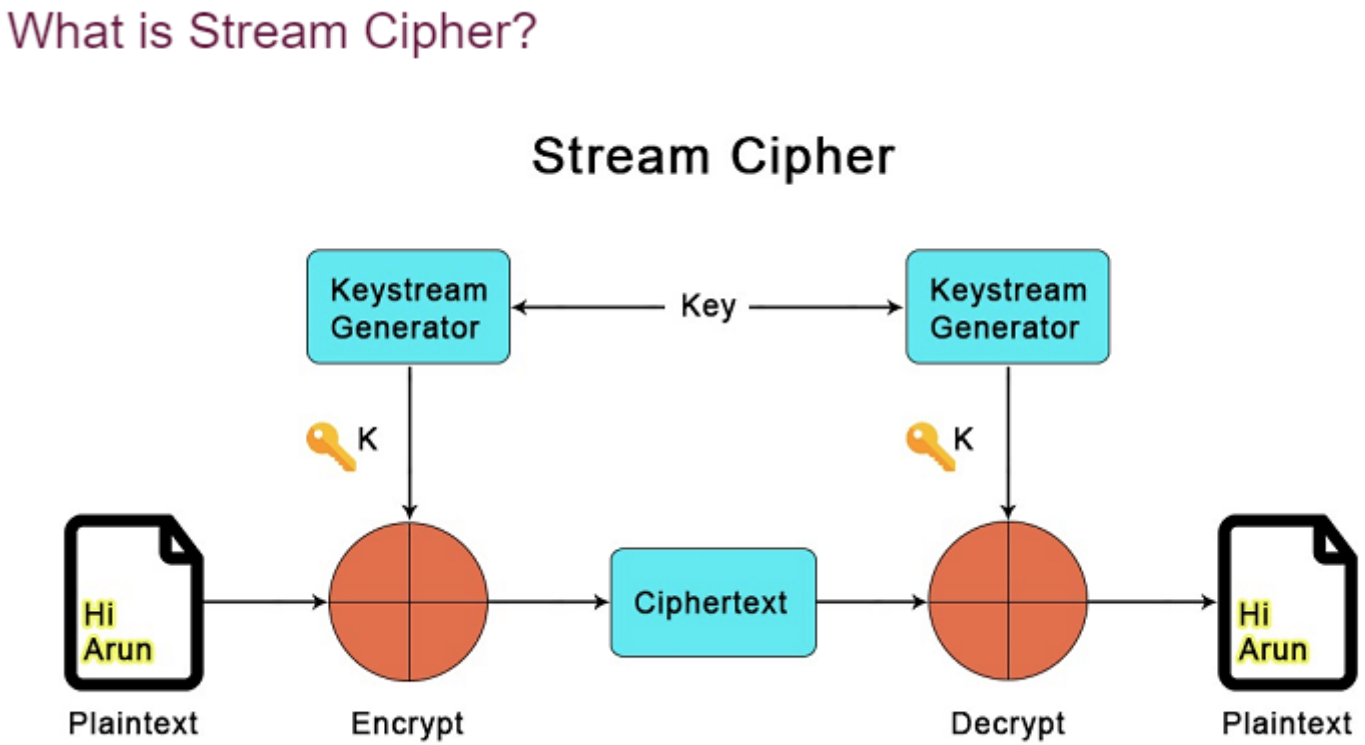


(a) Stream Cipher Using Algorithmic Bit Stream Generator



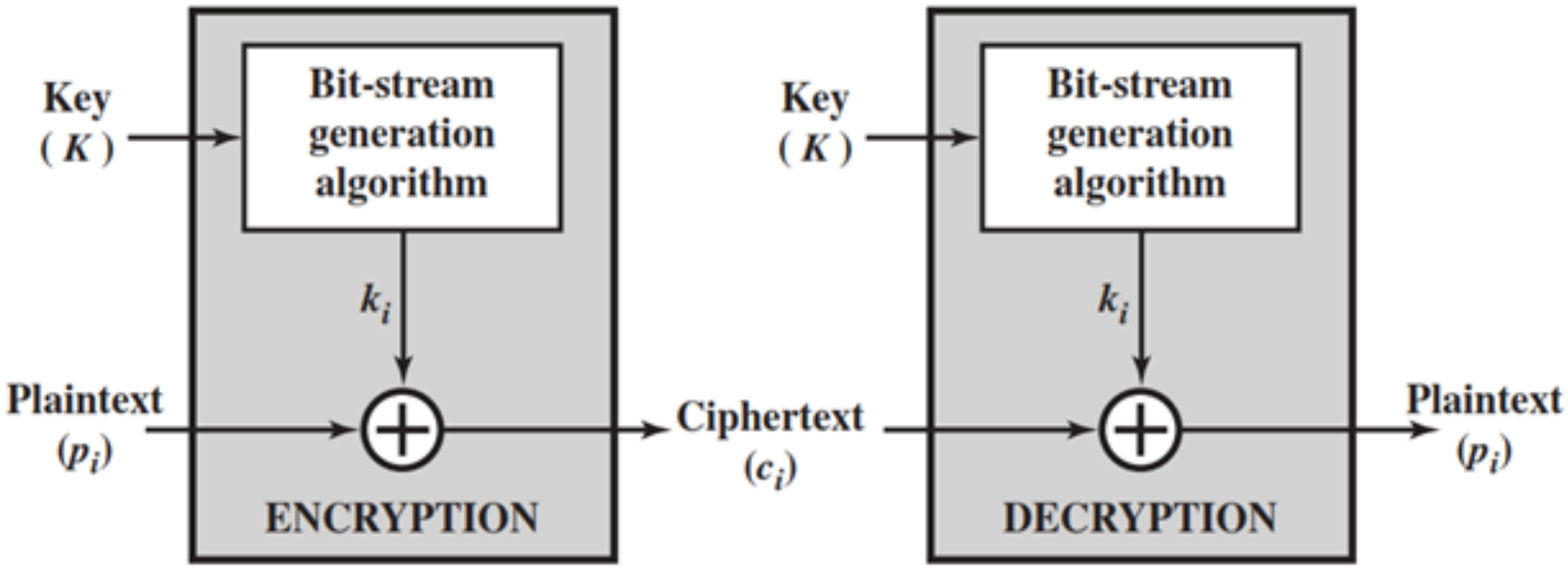
(b) Block Cipher

What is Stream Cipher?



Stream Cipher

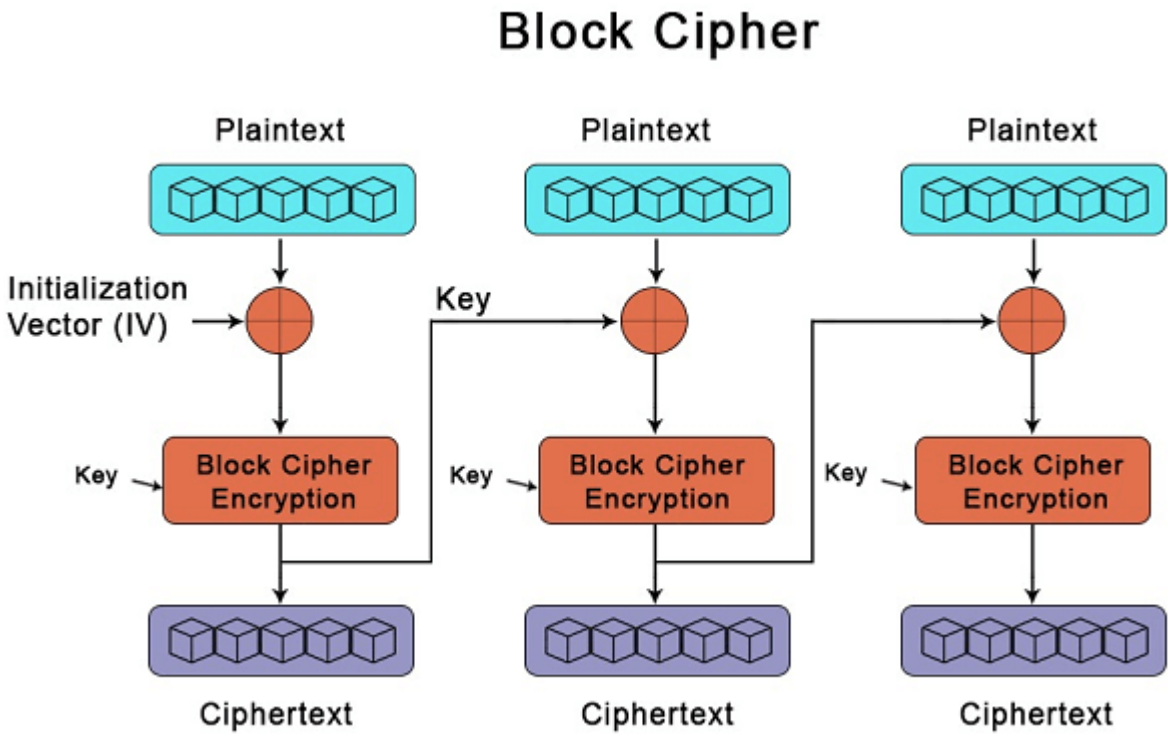
- A **stream cipher** is one that encrypts a digital data stream one bit or one byte at a time.
- Examples of classical stream ciphers are Autokeyed Vigenère cipher ,A5/1, RC4 and Vernam cipher.



What is Block Cipher?

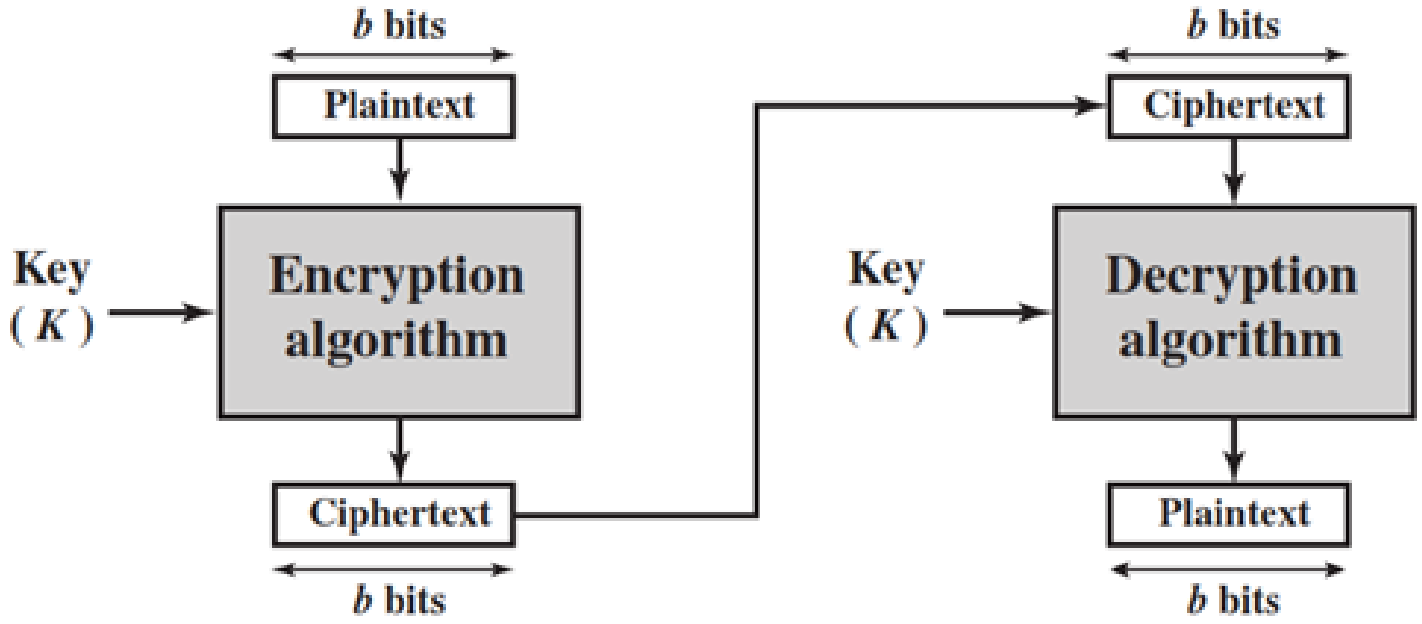
A **block cipher** is a ***symmetric cryptographic technique*** which we used to ***encrypt a fixed-size data block using a shared, secret key.*** During **encryption,** we used ***plaintext*** and **ciphertext** is the resultant encrypted text. It uses the same key to encrypt both the ***plaintext,*** and the ***ciphertext.***

A **block cipher** processes the data blocks of fixed size. Typically, a message's size exceeds a block's size. As a result, the lengthy message is broken up into a number of sequential message blocks, and the cipher operates on these blocks one at a time.



Block Cipher

- A **block cipher** is one in which a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length.
- Typically, a block size of **64 or 128** bits is used.
- Examples are Feistel Cipher, DES, Triple DES and AES

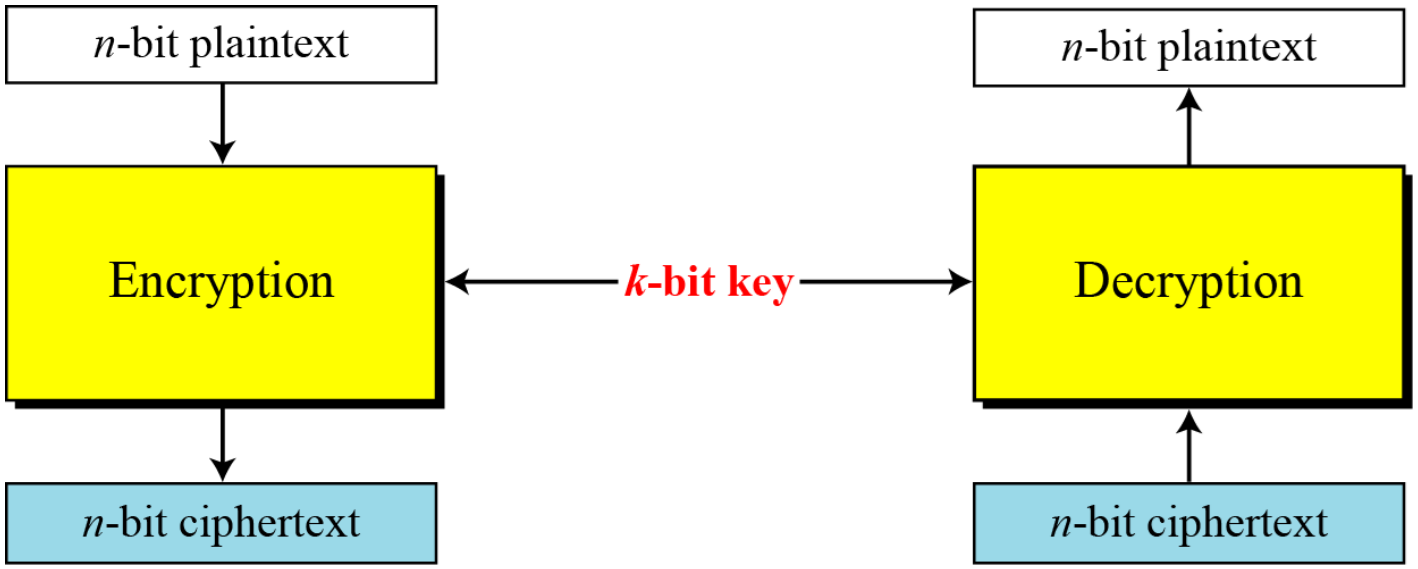


Parameters	Block Cipher	Stream Cipher
Definition	**Block Cipher** is the kind of encryption that converts plaintext by taking each block individually.	**Stream cipher** is the kind of encryption that converts plaintext by taking one byte of the plaintext at a time.
Principle	It uses both diffusion and confusion principles for the conversion (used later in encryption).	Only the confusion principle is used by Stream Cipher for the conversion.
Decryption	In **Block cipher, *reverse encryption or decryption*** is more difficult than stream cipher since more bits are combined to be encrypted in this scenario.	In a **stream cipher, XOR** is used for encryption that can quickly converted back to plain text.
Implementation	**Feistel Cipher** is the most popular block cipher implementation.	**Vernam Cipher** is the main implementation of Stream Cipher.
Implementation	Feistel Cipher is the most popular block cipher implementation.	Vernam Cipher is the main implementation of Stream Cipher.
Conversion of Bits	Since a block cipher converts blocks at once, it converts more significant bits than a stream cipher, which can convert 64 bits or more.	However, in stream cipher, only 8 bits can be transformed simultaneously.
Reversibility	It is difficult to reverse encrypted text.	It uses XOR encryption, which is easily reversed to the plain text.
Confusion and Diffusion	Block Cipher uses both confusion and diffusion.	Stream cipher relies on confusion only.
Algorithm modes used	**ECB** (Electronic Code Book) **CBC** (Cipher Block Chaining)	**CFB** (Cipher Feedback) **OFB** (Output Feedback)
Complexity	Simple design	Complex comparatively
No of bits used	**64 Bits** or more	**8 Bits**

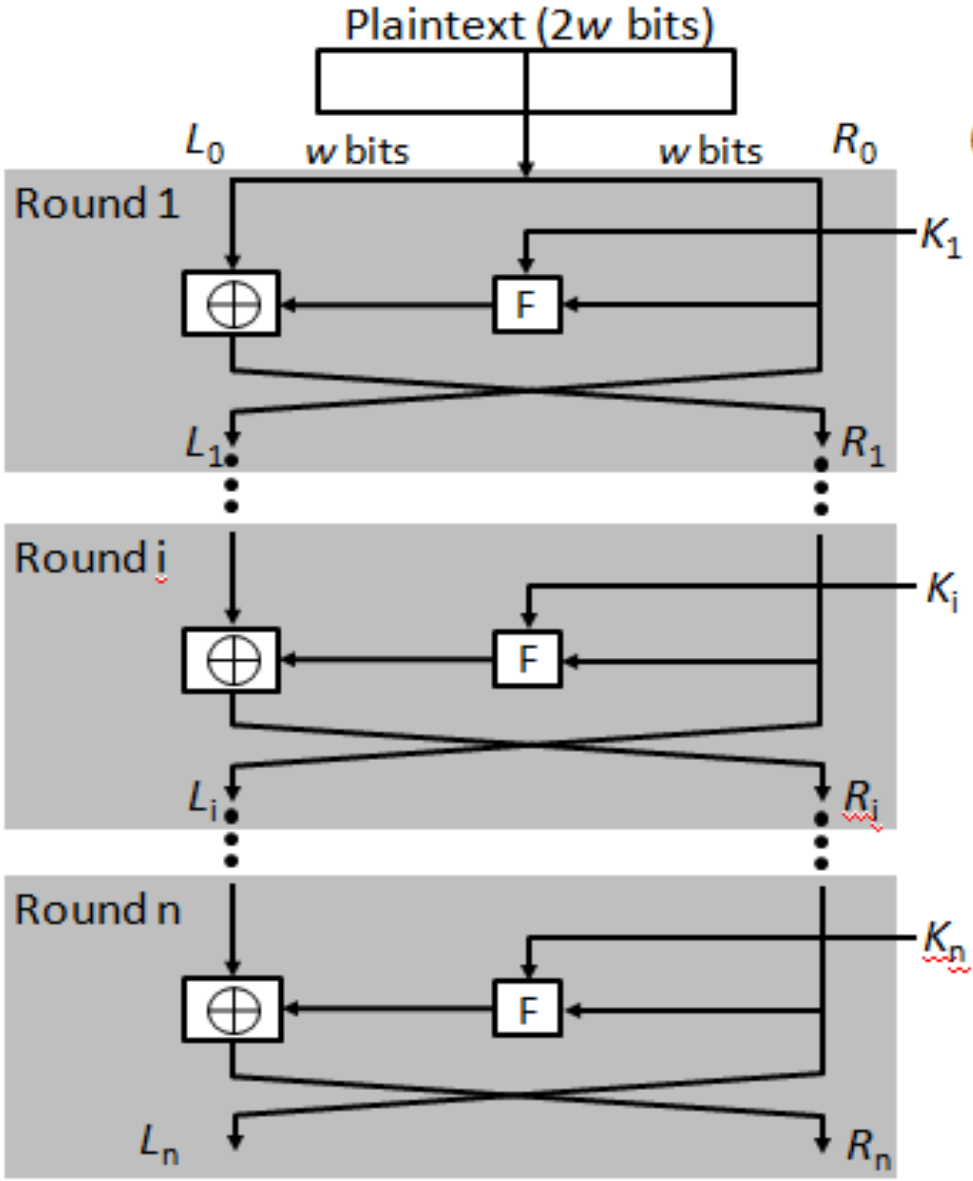
MODERN BLOCK CIPHERS

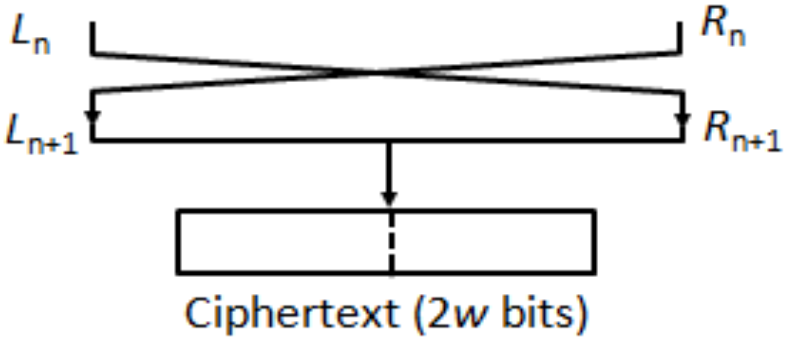
A symmetric-key modern block cipher encrypts an n-bit block of plaintext or decrypts an n-bit block of ciphertext.

The encryption or decryption algorithm uses a k-bit key.



Feistel Cipher Structure
Or Block Cipher Structure





Feistel Cipher Structure

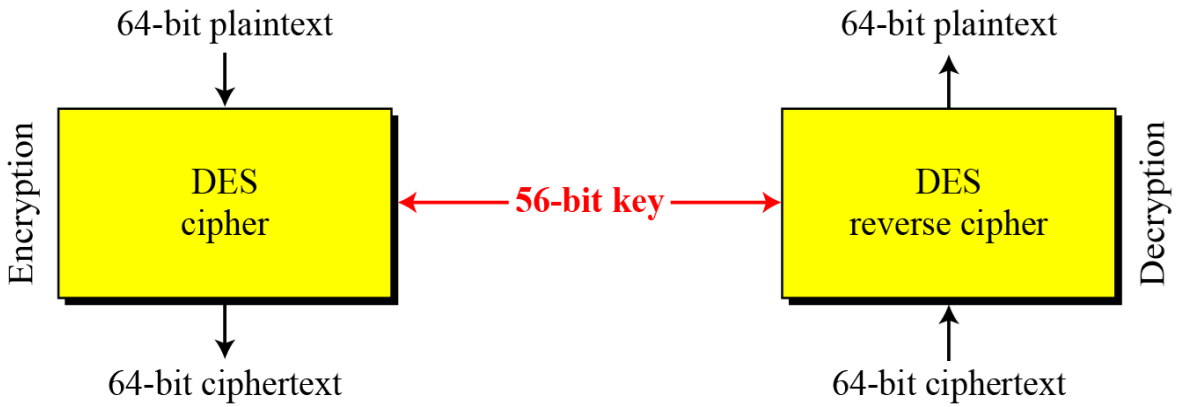
- Input plaintext block of length 2w bits
- key K = n bits , Sub-keys: K_1, K_2, …, K_n (Derived from K)
- All rounds have the same structure.
- A **substitution** is performed by taking exclusive-OR on left half(Li) of the data and the output of round function F which has inputs right half(Ri) and sub key ki.
- A **permutation** is performed that consists of interchange of two halves of data.
- This structure is called **Substitution-Permutation Network** (SPN)

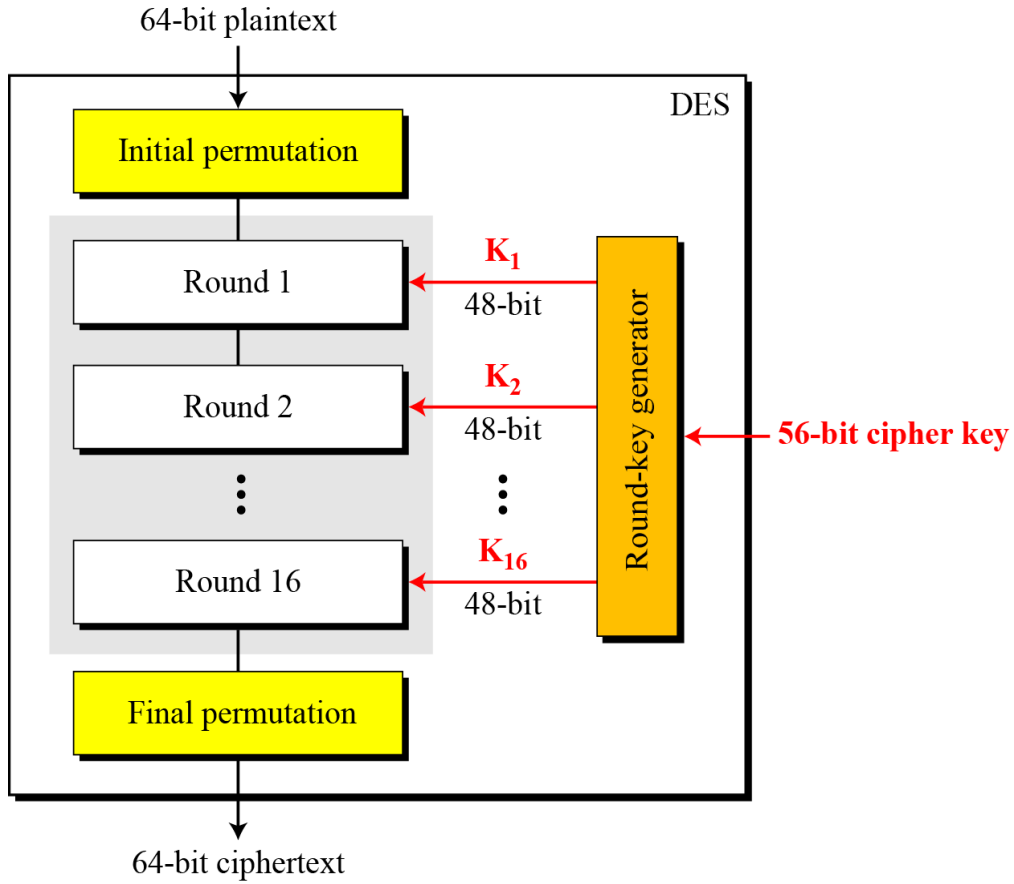
Feistel Network Factors

- **Block size:** Common block size of 64-bit. However, the new algorithms uses a 128-bit, 256-bit block size.
- **Key size:** Key sizes of 64 bits or less are now widely considered to be insufficient, These days at least 128 bit, more better, e.g. 192 or 256 bit
- **Number of rounds:** A typical size is 16 rounds.
- **Round function F:** Again, greater complexity generally means greater resistance to cryptanalysis.
- **Subkey generation algorithm:** Greater complexity in this algorithm should lead to greater difficulty of cryptanalysis.

Data Encryption Standard (DES)

- Type: Block Cipher
- Block Size : 64-bit
- Key Size: 64-bit, with only 56-bit effective
- Number of Rounds: 16





Encryption and decryption with DES

Asymmetric-Key Cryptography

Note

Symmetric-key cryptography is based on sharing secrecy; asymmetric-key cryptography is based on personal secrecy.

Symmetric-Key Cryptography

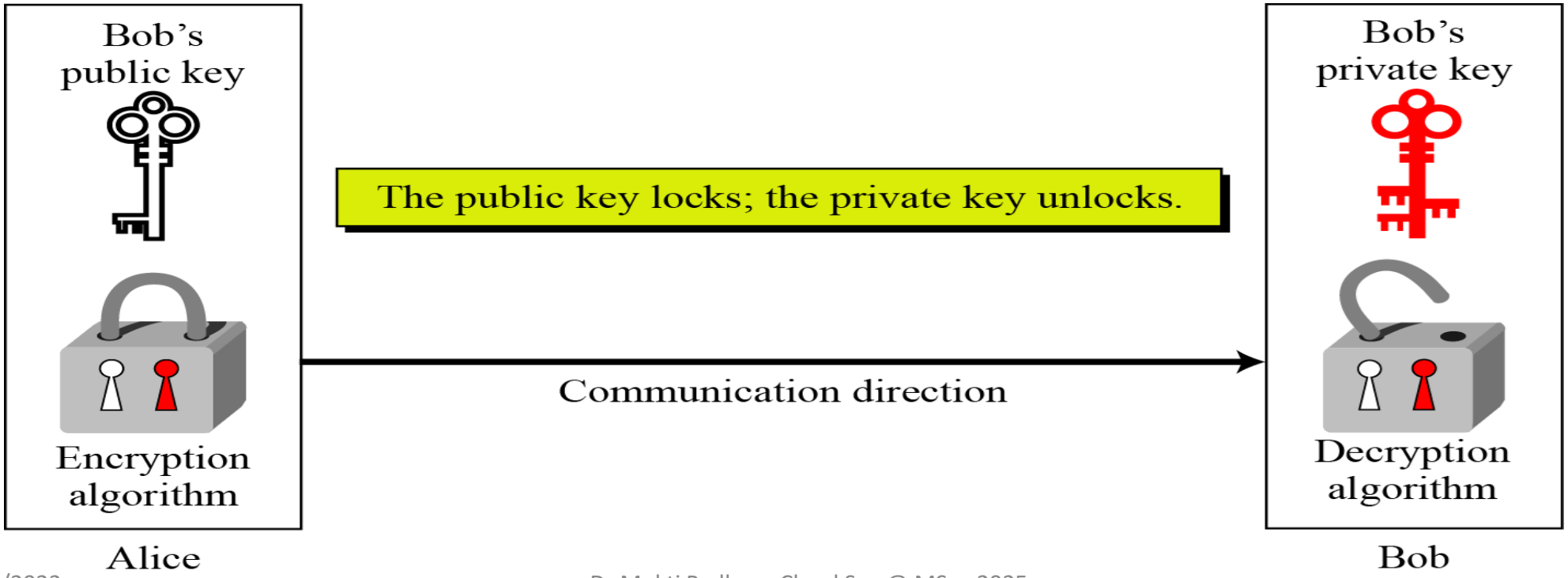
- traditional **private/secret/single key** cryptography uses **one** key
- shared by both sender and receiver
- if this key is disclosed communications are compromised
- also is **symmetric**, parties are equal
- hence does not protect sender from receiver forging a message & claiming is sent by sender

Asymmetric-Key Cryptography

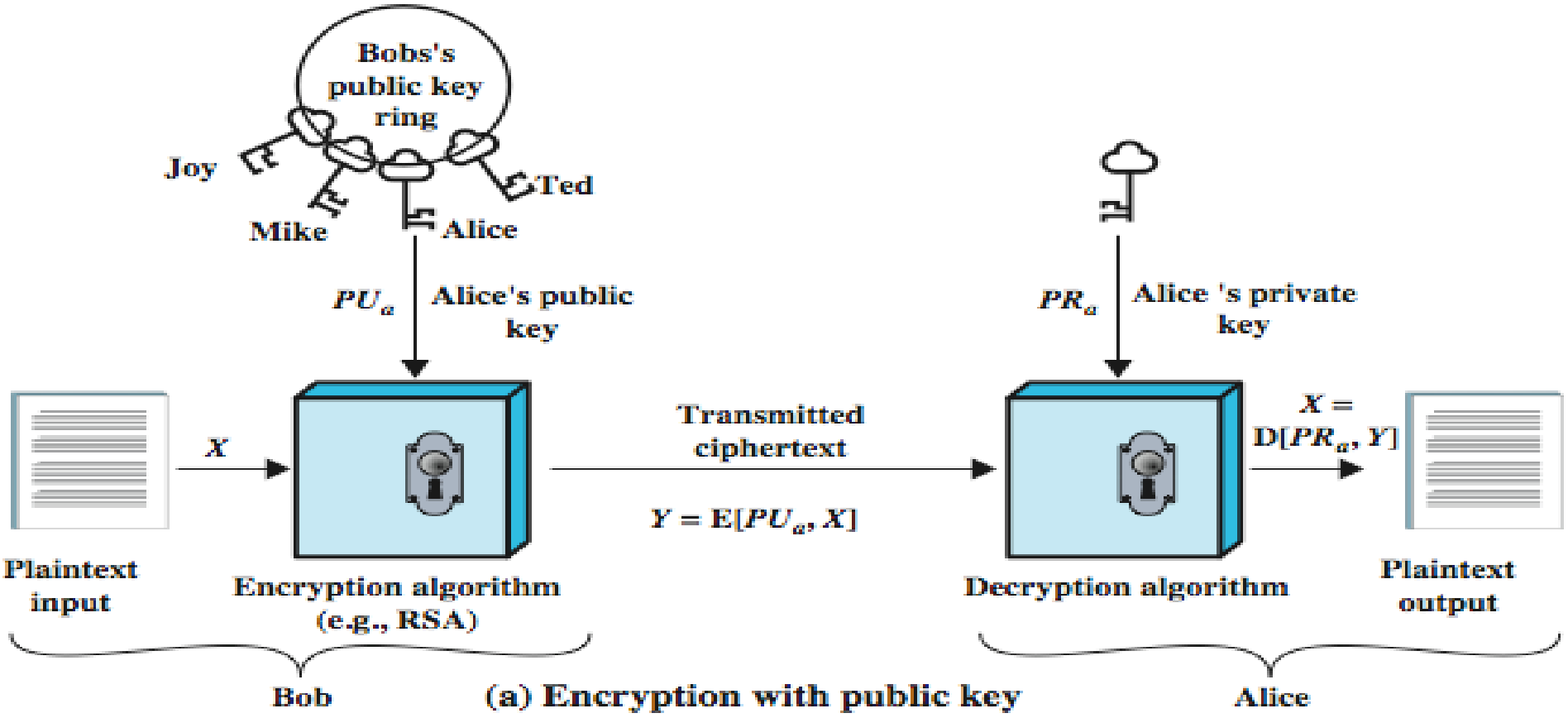
- probably most significant advance in the 3000 year history of cryptography
- uses **two** keys – a public & a private key
- **public-key/two-key/asymmetric** cryptography involves the use of **two** keys:
  - a **public-key**, which may be known by anybody, and can be used to **encrypt messages**, and **verify signatures**
  - a related **private-key**, known only to the recipient, used to **decrypt messages**, and **sign** (create) **signatures**
- **infeasible to determine private key from public**
- is **asymmetric** because
  - those who encrypt messages or verify signatures **cannot** decrypt messages or create signatures

Asymmetric key cryptography uses two separate keys: one private and one public.

Locking and unlocking in asymmetric-key cryptosystem



Asymmetric/Public-Key Cryptography Many to one communication



(a) Encryption with public key

Why Public-Key Cryptography?

- developed to address two key issues:
  - **key distribution** – how to have secure communications in general without having to trust a KDC with your key
  - **digital signatures** – how to verify a message comes intact from the claimed sender
- public invention due to Whitfield Diffie & Martin Hellman at Stanford University in 1976
  - known earlier in classified community

Public-Key Applications

- can classify uses into 3 categories:
  - **encryption/decryption** (provide secrecy)
  - **digital signatures** (provide authentication)
  - **key exchange** (of session keys)
- some algorithms are suitable for all uses, others are specific to one

Algorithm	Encryption/Decryption	Digital Signature	Key Exchange
RSA	Yes	Yes	Yes
Elliptic Curve	Yes	Yes	Yes
Diffie-Hellman	No	No	Yes
DSS	No	Yes	No

Symmetric-key Distribution

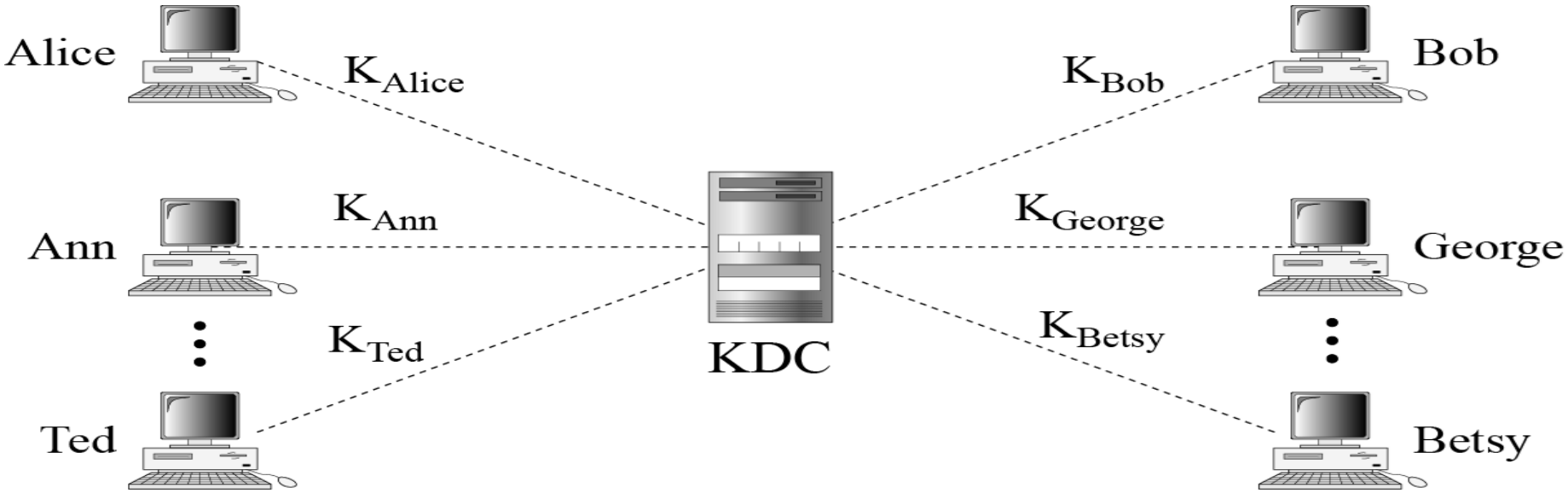
Symmetric-key cryptography is more efficient than asymmetric-key cryptography for enciphering large messages. Symmetric-key cryptography, however, needs a shared secret key between two parties. The distribution of keys is another problem.

Two Possible Ways:

1. Key-Distribution Center: KDC
2. Session Keys

Key-Distribution Center: KDC

Key-distribution center (KDC)



Symmetric-key Agreement

Alice and Bob can create a session key between themselves without using a KDC. This method of session-key creation is referred to as the symmetric-key agreement.

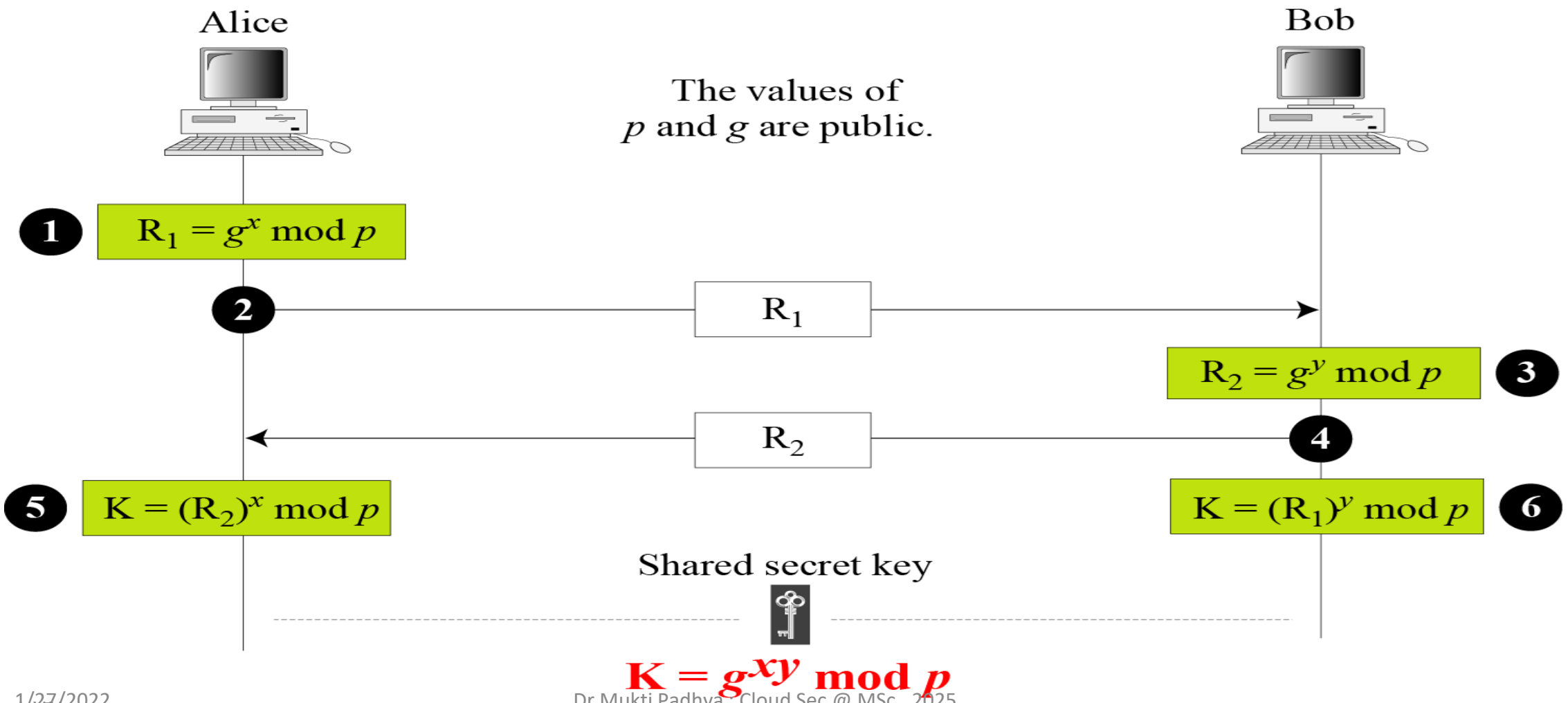
<u>Secure Techniques for Key Agreement:</u>

1. Diffie-Hellman Key Agreement
2. Station-to-Station Key Agreement

Diffie-Hellman key Exchange

- The purpose of the **Diffie-Hellman** algorithm is to enable two users to securely exchange a key that can be used for subsequent encryption of message.
- This algorithm depends for its effectiveness on the difficulty of computing **discrete logarithms**.

Diffie-Hellman Key Agreement



Primitive root

- Let p be a prime number
- Then a is a primitive root for p, if the powers of a modulo p generates all integers from **1 to $p-1$** in some permutation.

$$a \bmod p, a^2 \bmod p, \ldots, a^{p-1} \bmod p$$

- Example: p = 7 then primitive root is 3 because powers of 3 mod 7 generates all the integers from 1 to 6

$$3^1 = 3 \equiv 3 \pmod 7$$
$$3^2 = 9 \equiv 2 \pmod 7$$
$$3^3 = 27 \equiv 6 \pmod 7$$
$$3^4 = 81 \equiv 4 \pmod 7$$
$$3^5 = 243 \equiv 5 \pmod 7$$
$$3^6 = 729 \equiv 1 \pmod 7$$

Example

Let us give a trivial example to make the procedure clear. Our example uses small numbers, but note that in a real situation, the numbers are very large. Assume that $g = 7$ and $p = 23$. The steps are as follows:

1. Alice chooses $x = 3$ and calculates $R_1 = 7\text{^}3 \bmod 23 = 21$.
2. Bob chooses $y = 6$ and calculates $R_2 = 7\text{^}6 \bmod 23 = 4$.
3. Alice sends the number 21 to Bob.
4. Bob sends the number 4 to Alice.
5. Alice calculates the symmetric key K = 4^3 mod 23 = 18.
6. Bob calculates the symmetric key K = 21^6 mod 23 = 18.
7. The value of K is the same for both Alice and Bob; $g^{xy} \bmod p = 7\text{^}18 \bmod 35 = 18$.

Example

Let us give a more realistic example. We used a program to create a random integer of 512 bits (the ideal is 1024 bits). The integer p is a 159-digit number. We also choose g, x, and y as shown below:

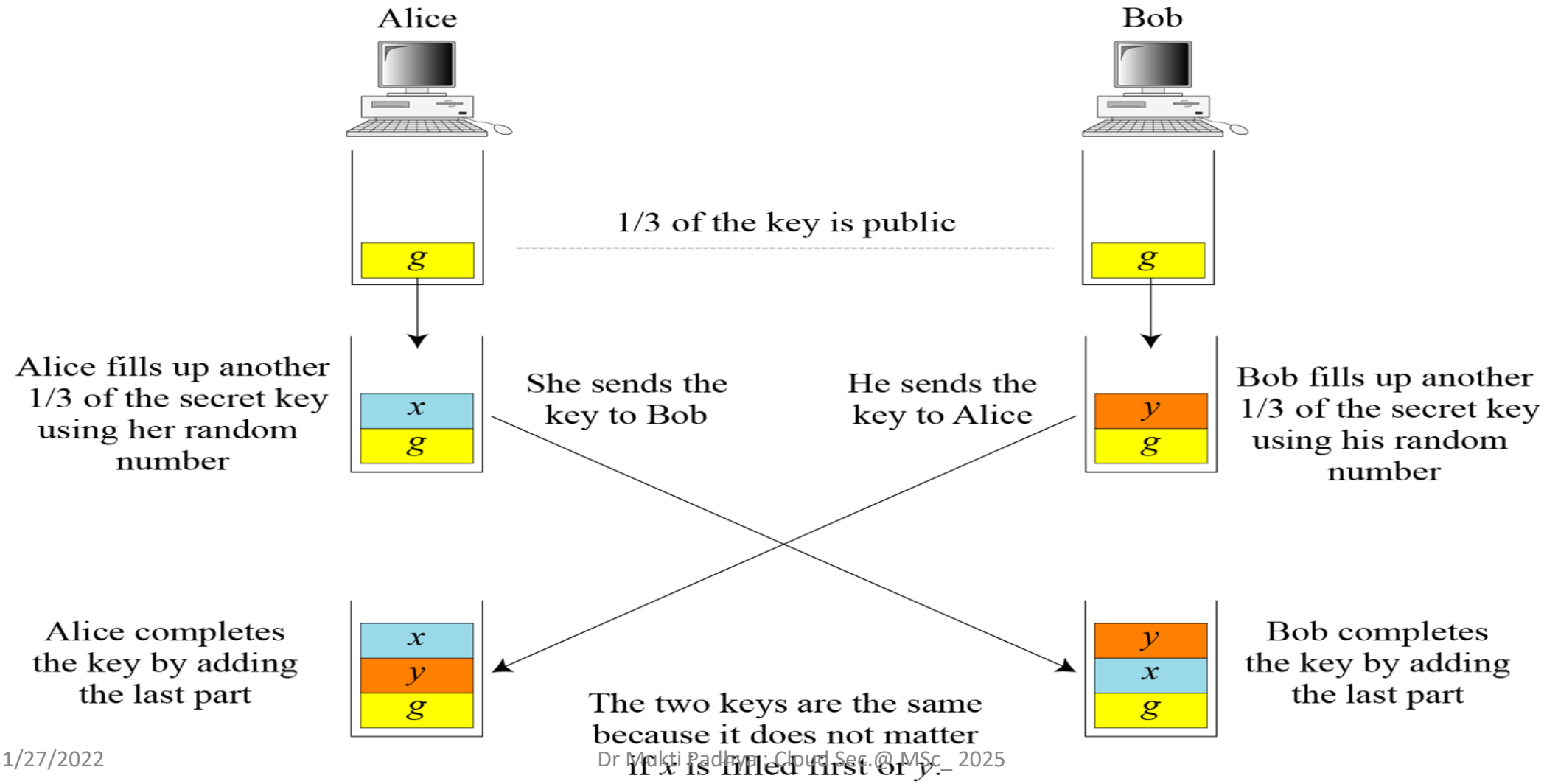
p	7646242985634935721824937659550305074763380967269497489235737728609252356666607554236374233096611800333381061947301309504147387009991780436548785807987581
g	2
x	557
y	273

Continued

The following shows the values of R_1, R_2, and K.

R_1	844920284205665505216172947491035094143433698520012660862863631067673619959280828586700802131859290945140217500319973312945836083821943065966020157955354
R_2	435262838709200379470747114895581627636389116262115557975123379218566310011435718208390040181876486841753831165342691630263421106721508589625520128859414 3
K	155638000664522290596225827523270765273218046944423678520320400146406500887936651204257426776608327911017153038674561252213151610976584200120408643361774 0

Diffie-Hellman idea



Example:

Step 1: Alice and Bob get public numbers P = 23, G = 9

Step 2: Alice selected a private key x = 4 and Bob selected a private key y = 3

Step 3: Alice and Bob compute public values

Alice: R1= ?

Bob: R2 = ?

Step 4: Alice and Bob exchange public numbers

Step 5: Alice receives public key R2 and Bob receives public key R1

Step 6: Alice and Bob compute symmetric keys

Alice: ka = ?

Bob: kb = ?

Step 7: What is the value of shared secret key k??.

Example:

Step 1: Alice and Bob get public numbers P = 23, G = 9

Step 2: Alice selected a private key x = 4 and Bob selected a private key y = 3

Step 3: Alice and Bob compute public values

Alice: R1 =(9^4 mod 23) = (6561 mod 23) = 6

Bob: R2 = (9^3 mod 23) = (729 mod 23) = 16

Step 4: Alice and Bob exchange public numbers

Step 5: Alice receives public key R2 =16 and Bob receives public key R1 = 6

Step 6: Alice and Bob compute symmetric keys

Alice: ka = R2^a mod p = 65536 mod 23 = 9

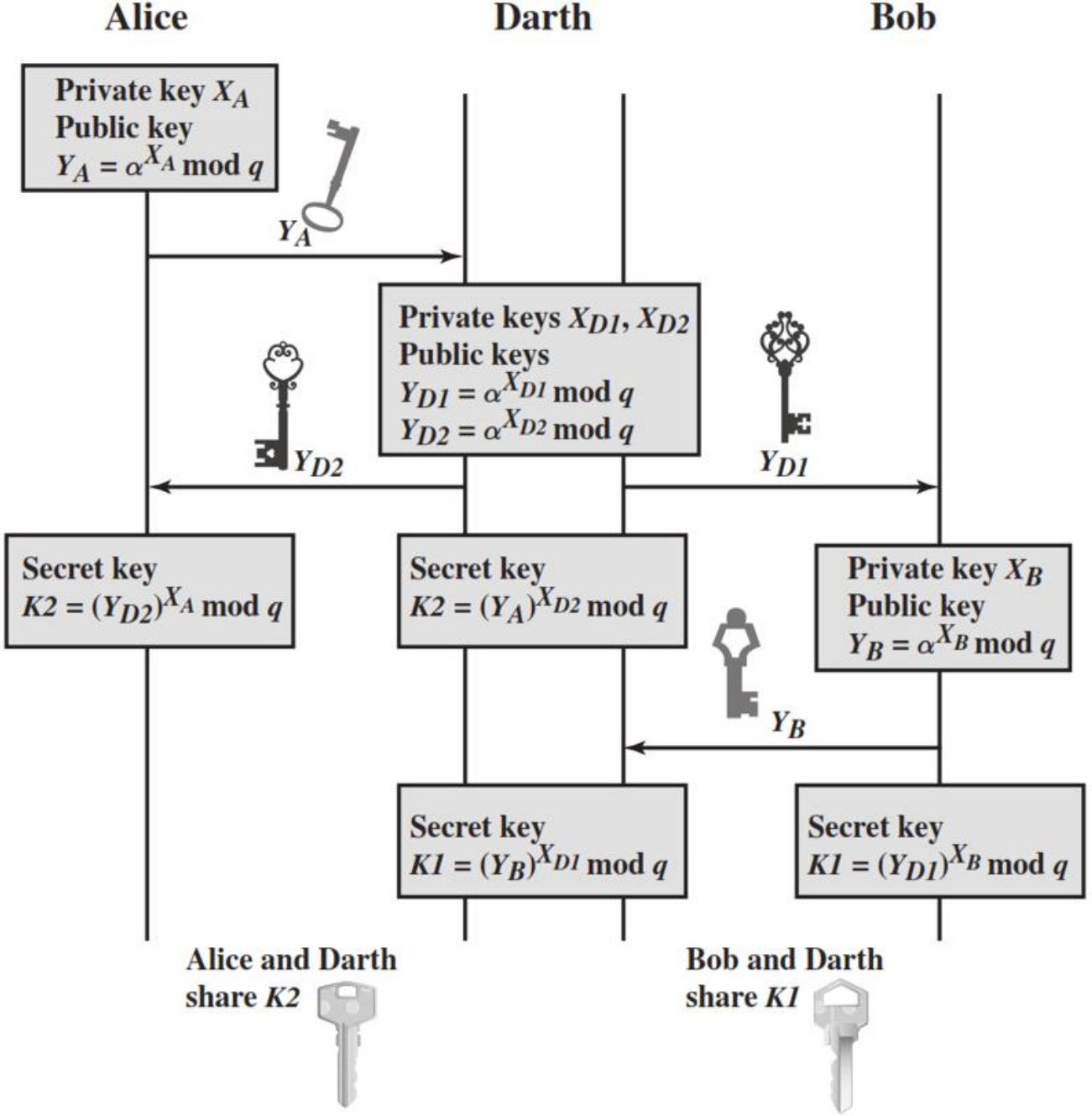
Bob: kb = R1^b mod p = 216 mod 23 = 9

Step 7: K=9 is the shared secret.

Man in the middle attack

- Suppose Alice and Bob wish to exchange keys, and Darth is the adversary.

1. Darth prepares for the attack by generating two random private keys X_{D1} and X_{D2} and then computes corresponding public keys Y_{D1} and Y_{D2}.
2. Alice transmits Y_A to Bob.
3. Darth intercepts Y_A and transmits Y_{D1} to Bob. Darth also calculates $K_2=(Y_A)^{XD2} \bmod q$.
4. Bob receives Y_{D1} and calculates $K_1 = (Y_{D1})^{XB} \bmod q$.
5. Bob transmits Y_B to Alice.
6. Darth intercepts Y_B and transmits Y_{D2} to Alice. Darth calculates $K_1=(Y_B)^{XD1} \bmod q$.
7. Alice receives Y_{D2} and calculates $K_2 = (Y_{D2})^{XA} \bmod q$.


Alice
Darth
Bob
Private key X_A
Public key
$Y_A = \alpha^{X_A} \bmod q$
Y_A
Private keys X_{D1}, X_{D2}
Public keys
$Y_{D1} = \alpha^{X_{D1}} \bmod q$
$Y_{D2} = \alpha^{X_{D2}} \bmod q$
Y_{D2}
Y_{D1}
Secret key
$K2 = (Y_{D2})^{X_A} \bmod q$
Secret key
$K2 = (Y_A)^{X_{D2}} \bmod q$
Private key X_B
Public key
$Y_B = \alpha^{X_B} \bmod q$
Y_B
Secret key
$K1 = (Y_B)^{X_{D1}} \bmod q$
Secret key
$K1 = (Y_{D1})^{X_B} \bmod q$
Alice and Darth
share $K2$
Bob and Darth
share $K1$

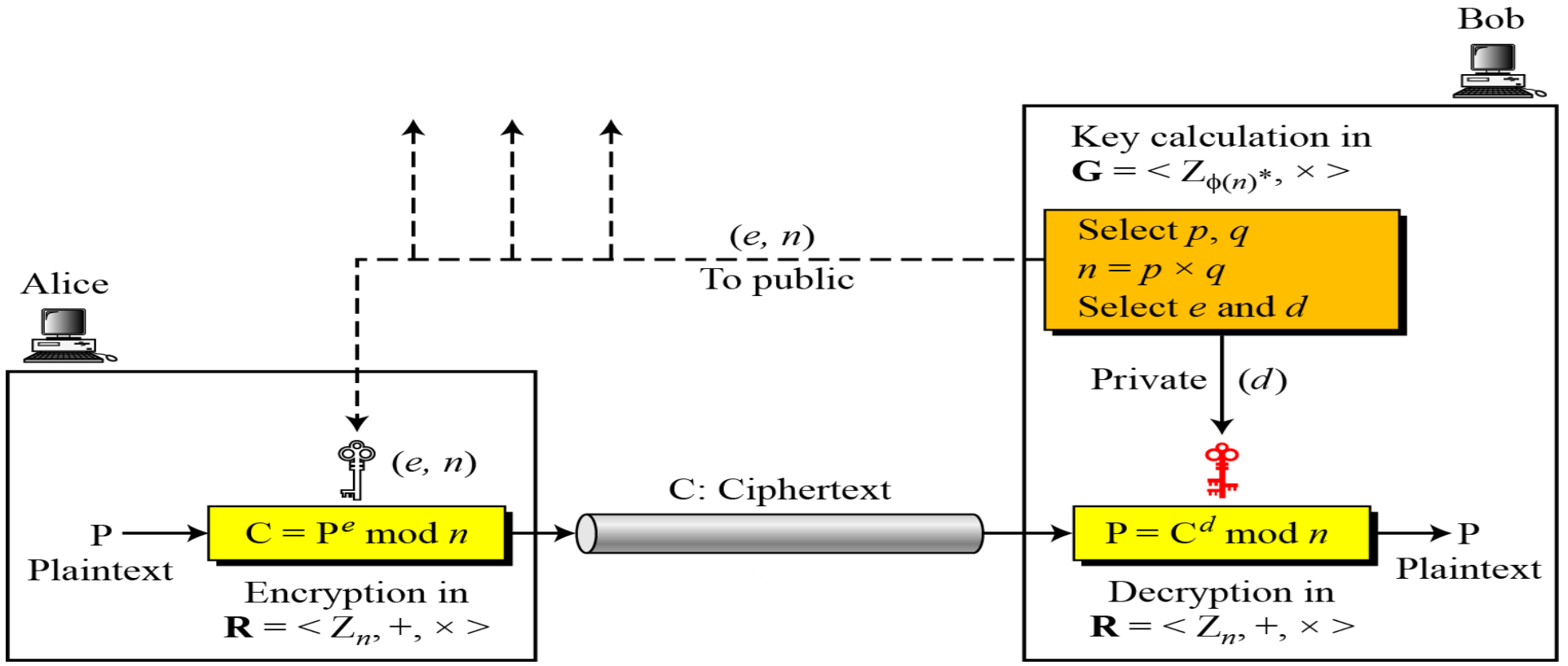
RSA

The most common public-key algorithm is the RSA cryptosystem, named for its inventors (Rivest, Shamir, and Adleman).

- by Rivest, Shamir & Adleman of MIT in 1977
- best known & widely used public-key scheme
- based on exponentiation in a finite (Galois) field over integers modulo a prime
  - nb. exponentiation takes $O((\log n)^3)$ operations (easy)
- uses large integers (eg. 1024 bits)
- security due to cost of factoring large numbers
  - nb. factorization takes $O(e^{\log n \log \log n})$ operations (hard)

Procedure

Encryption, decryption, and key generation in RSA



RSA Key Setup

- each user generates a public/private key pair by:
- selecting two large primes at random: p, q
- computing their system modulus $n=p.q$
  - note $\varnothing(n)=(p-1)(q-1)$
- selecting at random the encryption key e
  - where $1<e<\varnothing(n)$, $\gcd(e,\varnothing(n))=1$
- solve following equation to find decryption key d
  - $e.d=1 \bmod \varnothing(n)$ and $0 \leq d \leq n$
- publish their public encryption key: PU={e,n}
- keep secret private decryption key: PR={d,n}

Algorithm 10.2 *RSA Key Generation*

RSA_Key_Generation

{

Select two large primes p and q such that $p \neq q$.

$n \leftarrow p \times q$

$\phi(n) \leftarrow (p-1) \times (q-1)$

Select e such that $1 < e < \phi(n)$ and e is coprime to $\phi(n)$

$d \leftarrow e^{-1} \bmod \phi(n)$ // d is inverse of e modulo $\phi(n)$

Public_key $\leftarrow$ (e, n) // To be announced publicly

Private_key $\leftarrow d$ // To be kept secret

return Public_key and Private_key

}

RSA En/decryption

- to encrypt a message M the sender:
  - obtains **public key** of recipient $PU=\{e,n\}$
  - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- to decrypt the ciphertext C the owner:
  - uses their private key $PR=\{d,n\}$
  - computes: $M = C^d \bmod n$
- note that the message M must be smaller than the modulus n (block if needed)

Why RSA Works

- because of Euler's Theorem:
  - $a^{\phi(n)} \bmod n = 1$ where $\gcd(a,n)=1$
- in RSA have:
  - $n=p.q$
  - $\phi(n)=(p-1)(q-1)$
  - carefully chose e & d to be inverses $\bmod \phi(n)$
  - hence $e.d=1+k.\phi(n)$ for some k
- hence :

$$C^d = M^{e.d} = M^{1+k.\phi(n)} = M^1.(M^{\phi(n)})^k$$
$$= M^1.(1)^k = M^1 = M \bmod n$$

RSA Example - Key Setup

1. Select primes: $p=17$ & $q=11$
2. Calculate $n = pq = 17 \times 11 = 187$
3. Calculate $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e: $\gcd(e,160)=1$; choose $e=7$
5. Determine d: $de=1 \bmod 160$ and $d < 160$ Value is $d=23$ since $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish public key $PU=\{7,187\}$
7. Keep secret private key $PR=\{23,187\}$

RSA Example - En/Decryption

- sample RSA encryption/decryption is:
- given message $M = 88$ (nb. $88<187$)
- encryption:

$$C = 88^7 \bmod 187 = 11$$

- decryption:

$$M = 11^{23} \bmod 187 = 88$$

RSA Example

- Find n, ϕ(n), e, d for p=7 and q= 19 then demonstrate encryption and decryption for M = 6

Possible values for e: { 2, 3, 5}
Possible Values for d: {21,43,65 }

RSA Example

- Find n, ϕ(n), e, d for p=7 and q= 19 then demonstrate encryption and decryption for M = 6

n = p * q = 7 * 19 = 133

ϕ(n) = (p – 1) * (q – 1) = 108

Finding e relatively prime to 108
e = 2 => GCD(2, 108) = 2 (no)
e = 3 => GCD(3, 108) = 3 (no)
e = 5 => GCD(5, 108) = 1 (Yes)

- Finding d such that (d * e) mod ϕ(n) = 1
- We can rewrite this as d = (1 + j . ϕ(n)) / e

j = 0 => d = 1 / 5 = 0.2 ← integer ? (no)
j = 1 => d = 109 / 5 = 21.8 ← integer ? (no)
j = 2 => d = 217 / 5 = 43.4 ← integer ? (no)
j = 3 => d = 325 / 5 = 65 integer ? (yes)

Public key :
PU = { e, n } = {5, 133}
Private key :
PR = { d, n } = {65, 133}

RSA Example – cont…

- Encryption:

$C = M^e \bmod n$

$PU = \{ e, n \}, PU = \{ 5, 133 \}$

For message M = 6

$C = 6^5 \bmod 133$

$C = 7776 \bmod 33$

$C = 62$

- Decryption:

$M = C^d \bmod n$

$PR = \{ d, n \}, PU = \{ 65, 133 \}$

For C = 62

$M = 62^{65} \bmod 133$

$M = 2666 \bmod 33$

$M = 6$

RSA Example

- P and Q are two prime numbers. P=7, and Q=17. Take public key E=5. If plain text value is 10, then what will be cipher text value according to RSA algorithm?

RSA Example

- P and Q are two prime numbers. P=7, and Q=17. Take public key E=5. If plain text value is 10, then what will be cipher text value according to RSA algorithm?
- n = 119
- $\phi(n) = 96$
- e = 5
- d = 77
- PU = { 5, 119 }
- PR = {77, 119}
- $C = 10^5 \bmod 119 \Rightarrow C = 40$

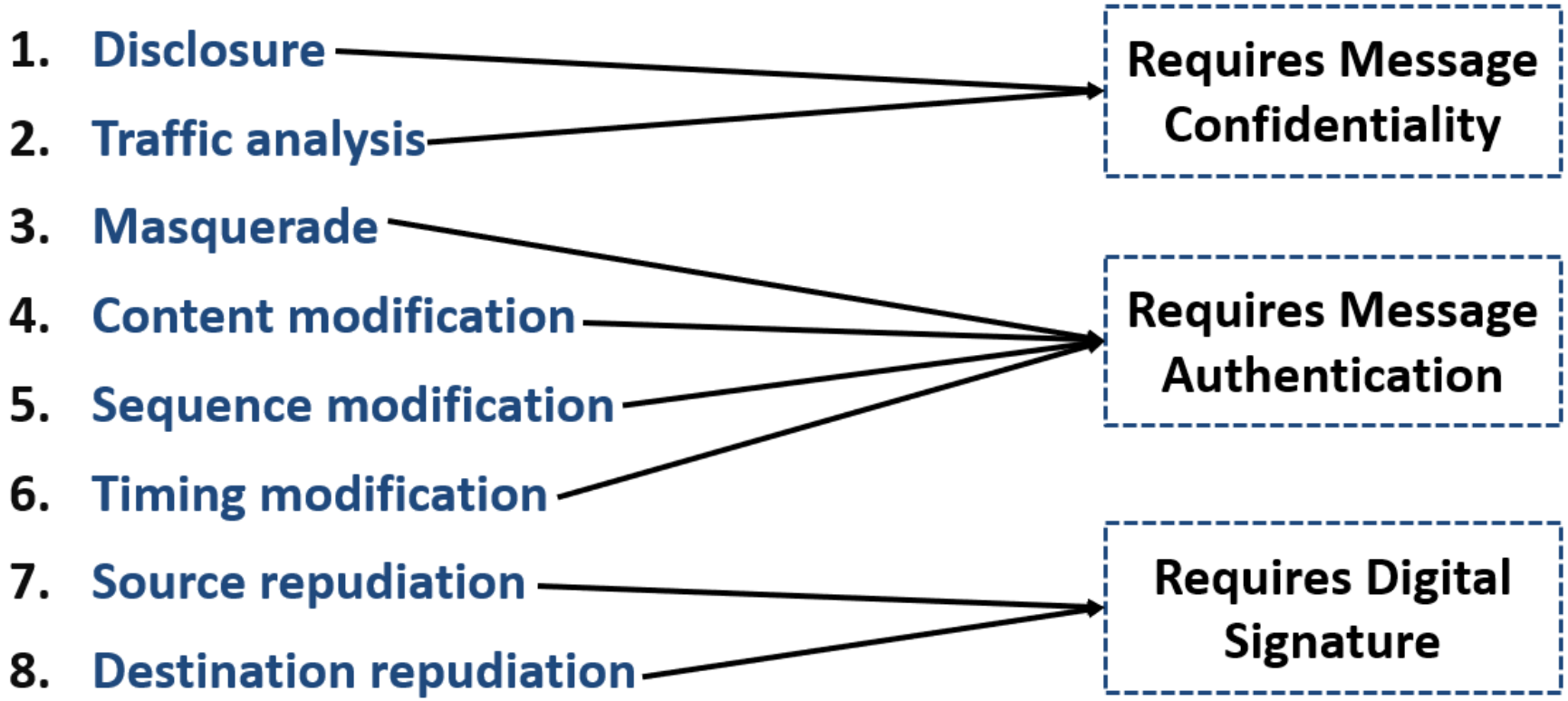
RSA Security

- possible approaches to attacking RSA are:
  - brute force key search - infeasible given size of numbers
  - mathematical attacks - based on difficulty of computing ø(n), by factoring modulus n
  - timing attacks - on running of decryption
  - chosen ciphertext attacks - given properties of RSA
- mathematical approach takes 3 forms:
  - factor `n=p.q`, hence compute `ø(n)` and then `d` `(Factoring Problem)`
  - determine `ø(n)` directly and compute d
  - find d directly

Message Authentication

- **Message authentication** is a procedure to verify that received messages come from the genuine source and have not been altered.
- Message authentication may also verify sequencing and timeliness.
- Message authentication is a mechanism or service used to verify the **integrity of a message**.
- Message authentication assures that data received are exactly as sent (i.e., contain no modification, insertion, deletion, or replay).

Message Authentication Requirements

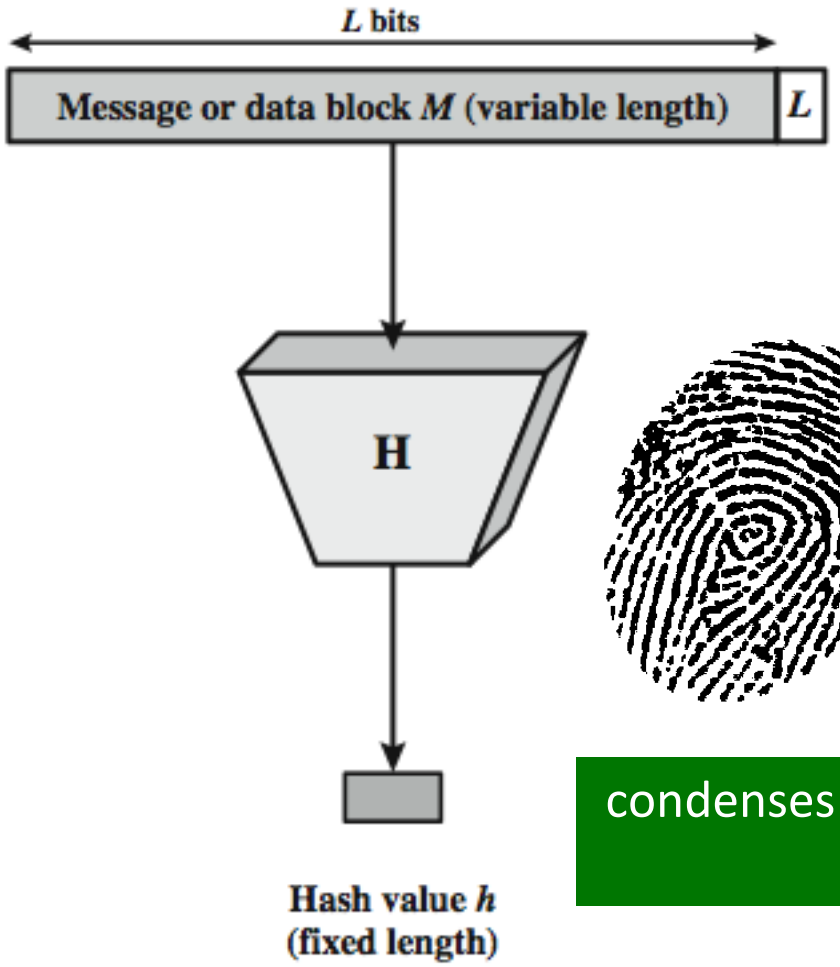


Motivation for Hash Algorithms

- Intuition
  - Limitation on non-cryptographic checksum
  - Very possible to construct a message that matches the checksum
- Goal
  - Design a code where the original message can not be inferred based on its checksum
  - such that an accidental or intentional change to the message will change the hash value

Dr. Mukti Padhya : Cloud Sec @ MSc_2023

Hash Function



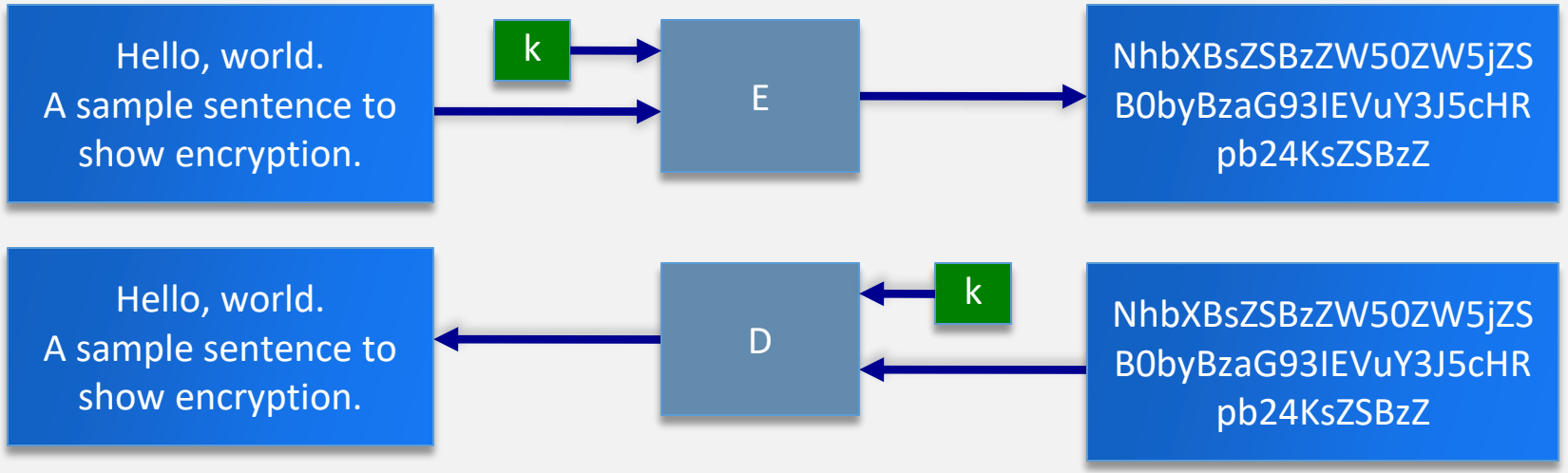
- The hash value represents concisely the longer message
  - may called the *message digest*
- A message digest is as a ``digital fingerprint'' of the original document



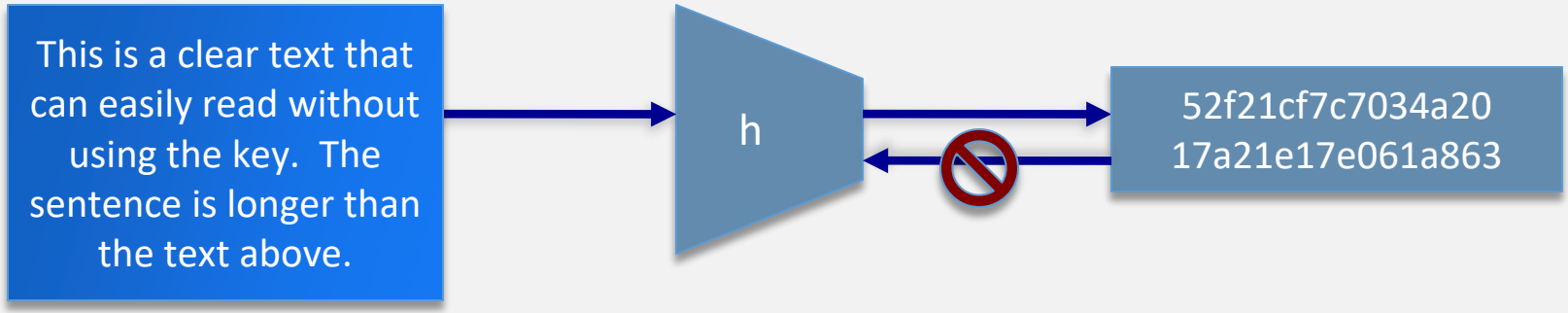
condenses arbitrary message to fixed size

$$h = H(M)$$

Hashing V.S. Encryption



- Encryption is two way, and requires a key to encrypt/decrypt



- Hashing is one-way. There is no 'de-hashing'

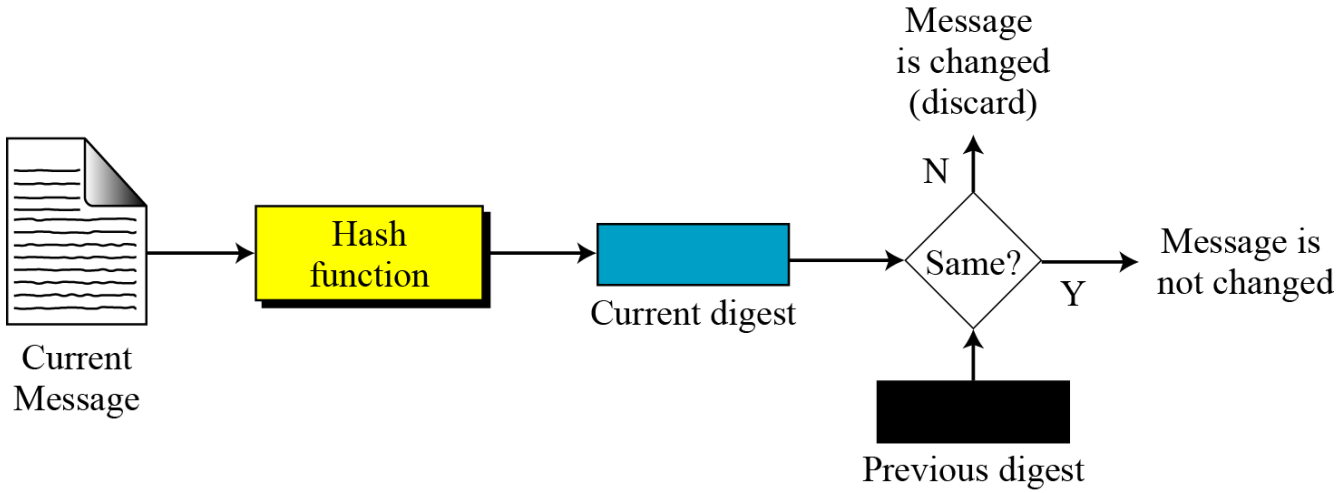
Hash Function Applications

- Used Alone
  - Fingerprint -- file integrity verification, public key fingerprint
  - Password storage (one-way encryption)

- Combined with encryption functions
  - Hash based Message Authentication Code (HMAC)
    - protects both a message's integrity and confideltaility
  - Digital signature
    - Ensuring Non-repudiation
    - Encrypt hash with private (signing) key and verify with public (verification) key

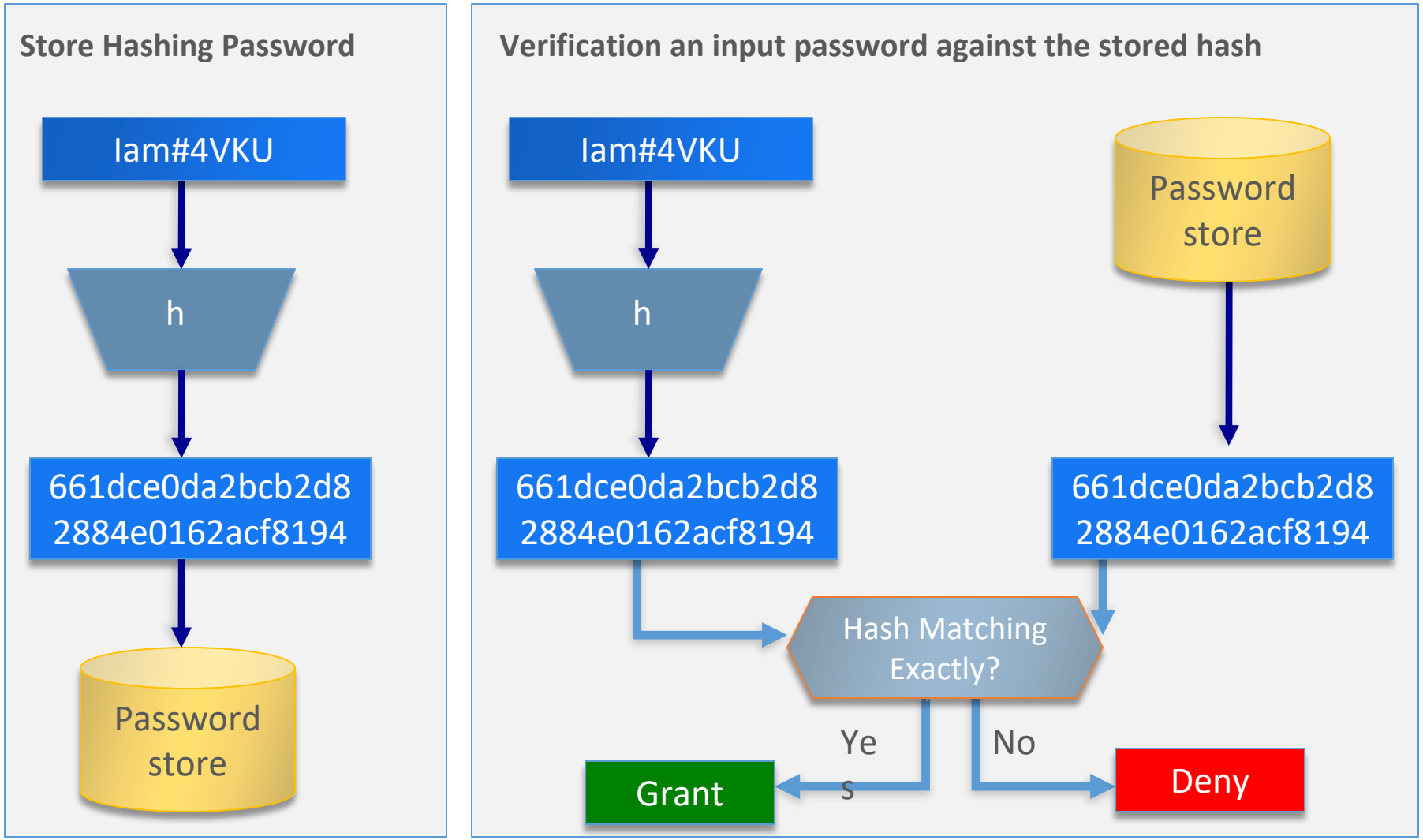
Dr. Mukti Padhya : Cloud Sec @ MSc_2023

Integrity



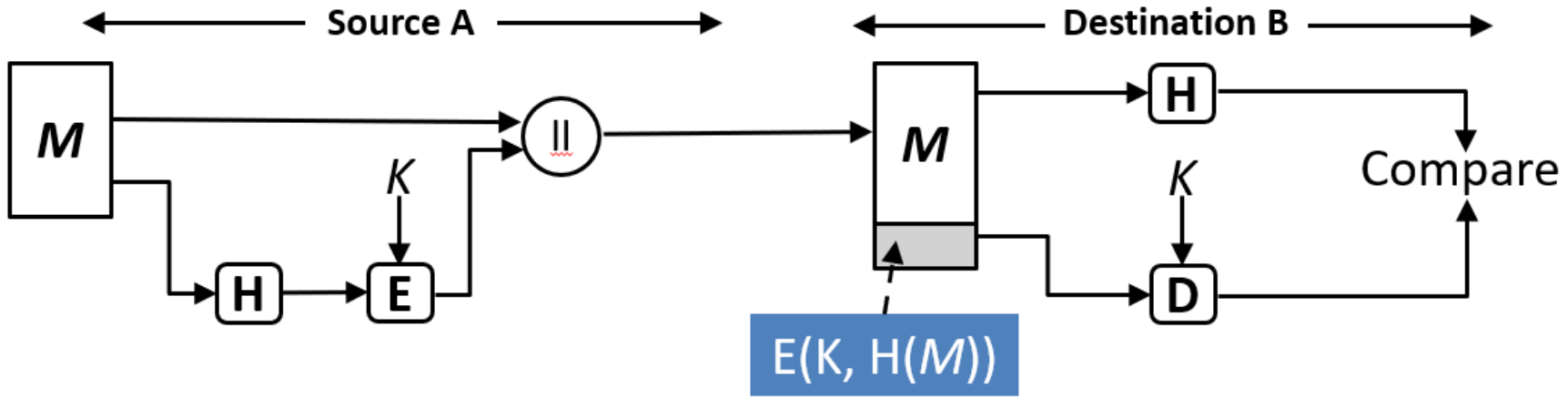
- to create a one-way password file
  - store hash of password not actual password
- for intrusion detection and virus detection
  - keep & check hash of files on system

Password Verification

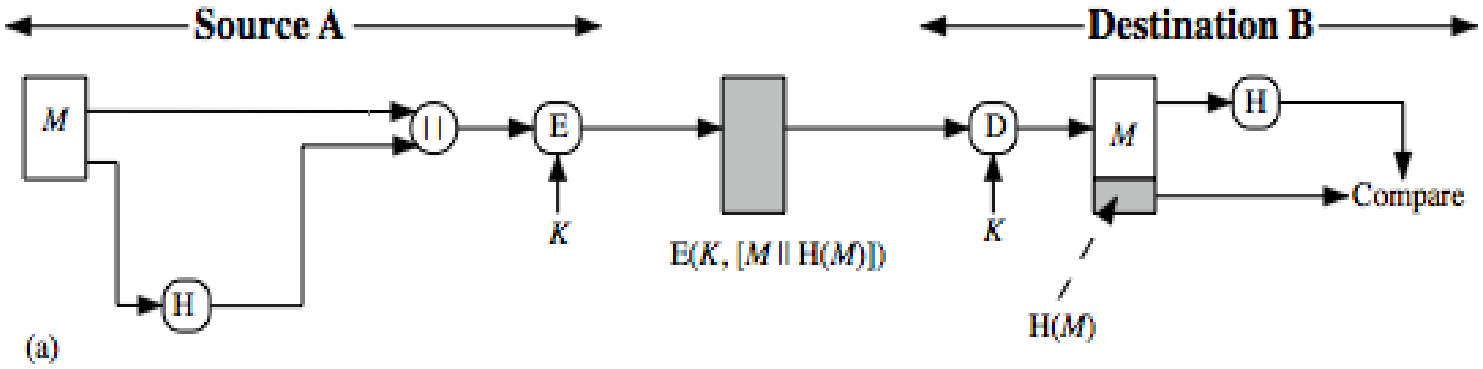


MAC (Message Authentication Code)

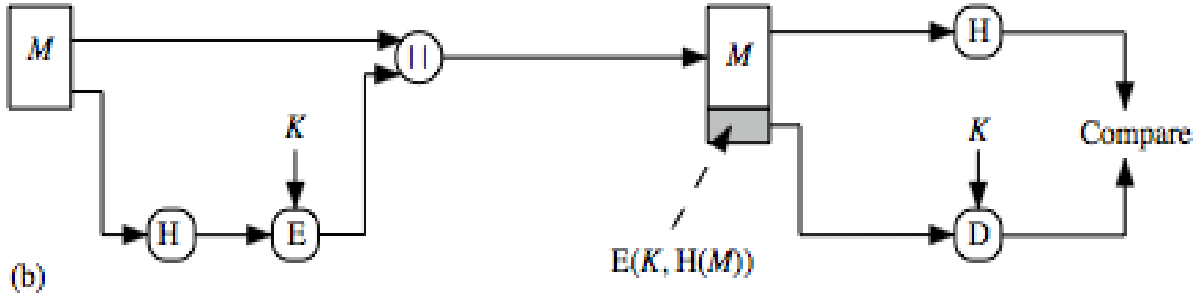
- More commonly, message authentication is achieved using a **MAC** also known as **keyed hash function**.
- MACs are used between two parties that share a secret key to authenticate information exchanged between those parties.
- A **MAC** function takes as input a secret key and a data block and produces a hash value, referred to as the **MAC**.



Hash Function Usages (I) : Message Authentication Methods

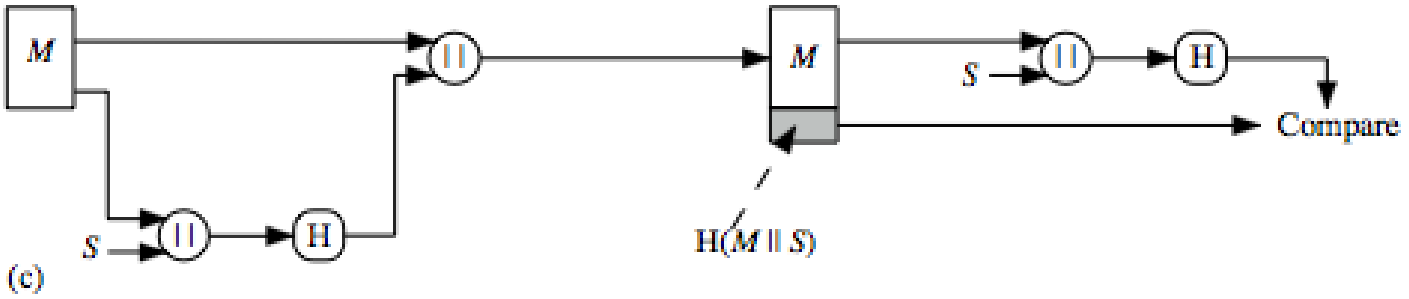


Message encrypted : Confidentiality and authentication

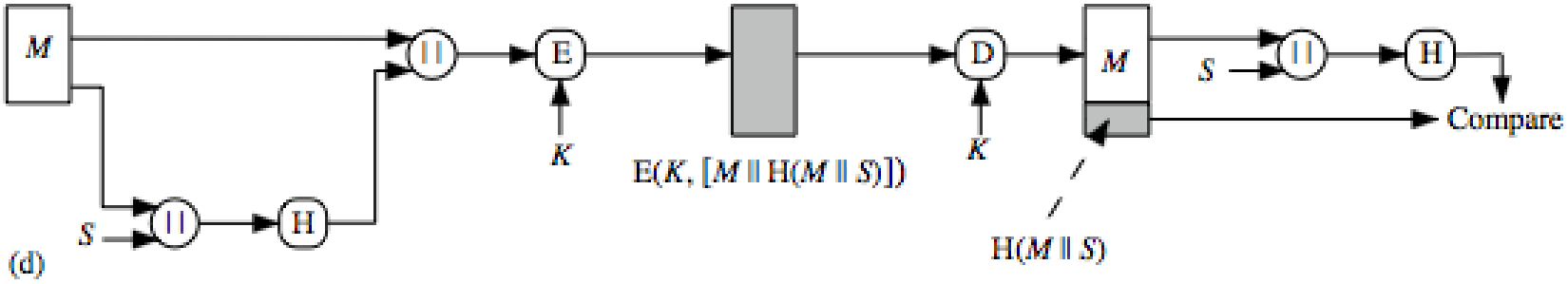


Message unencrypted: Authentication

Hash Function Usages (II) : Message Authentication Methods

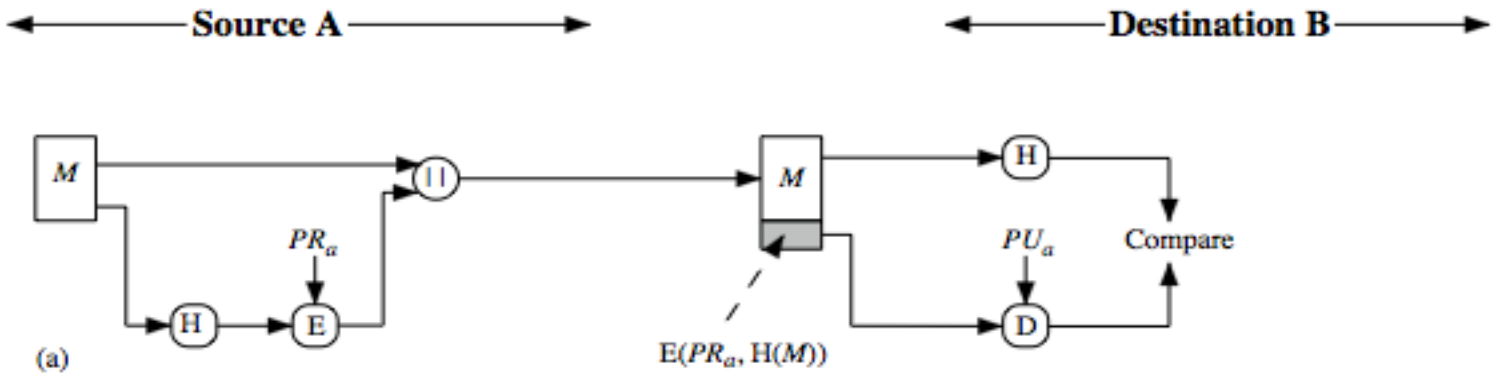


Message encrypted : Authentication (no encryption needed!)

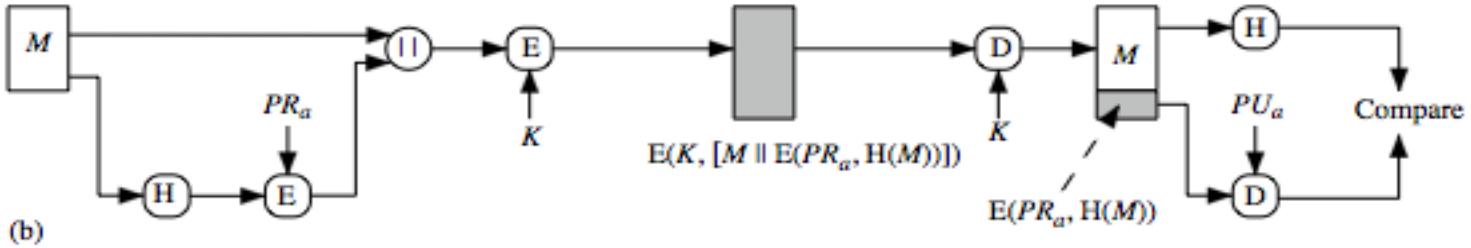


Message unencrypted: Authentication, confidentiality

Hash Function Usages (III) : Message Authentication Methods



Authentication, digital signature



Authentication, digital signature, confidentiality

Requirements for hash functions

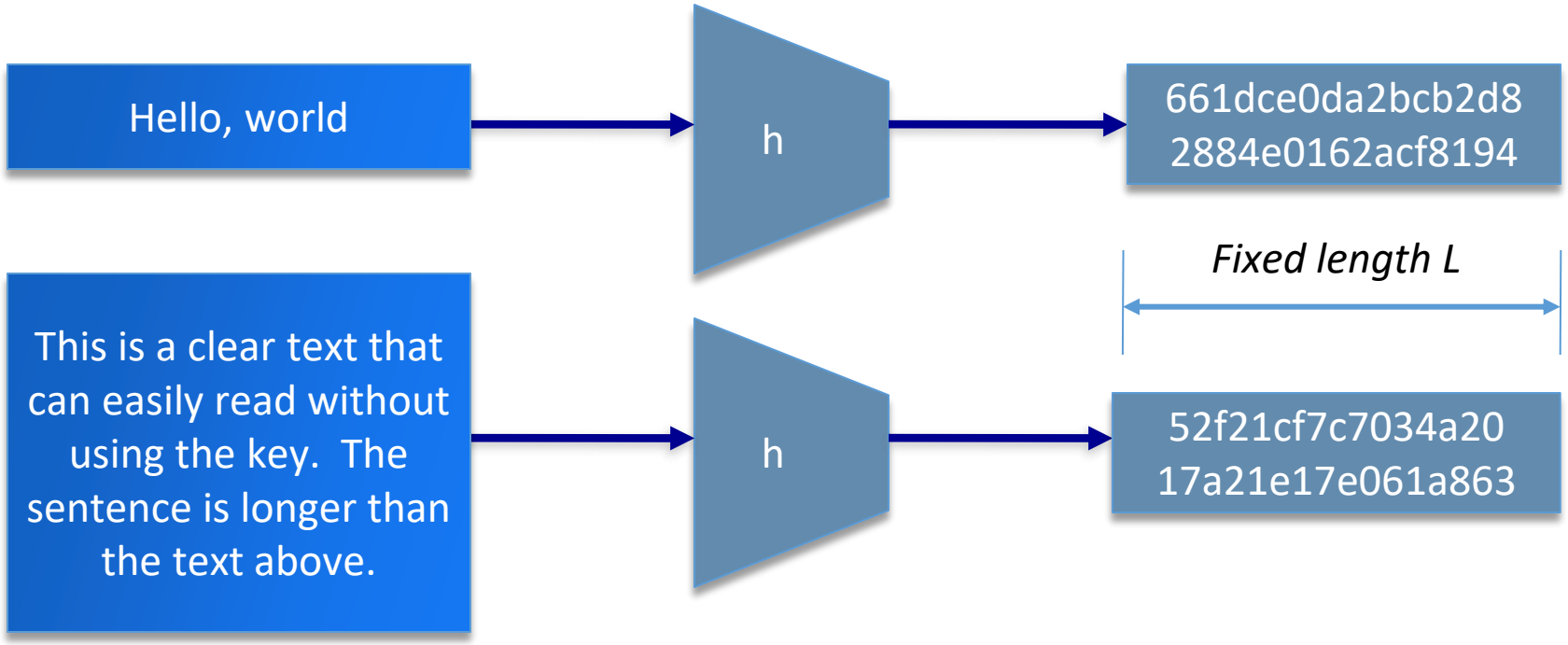
1. Can be applied to any *sized message M*.
2. Produces *fixed-length output h*.
3. It is *easy to compute h=H(M)* for any *message M*.
4. Given hash value *h* is *infeasible* to find *y* such that (*H(y) = h*)
   - *One-way property*
5. For given block *x*, it is computational infeasible to find

   $y \neq x$ with $H(y) = H(x)$
   - *Weak collision resistance*
6. It is computationally infeasible to find messages *m1* and *m2* with $H(m1) = H(m2)$
   - *Strong collision resistance*

Hash Function Properties

- Arbitrary-length message to fixed-length digest
- Preimage resistant (**One-way property**)
  - for a hash value *h = H(x),* we say that x is the **preimage** of ***h***
  - it is easy to generate a code given a message, but virtually impossible to generate a message given a code.
  - This property is important if the authentication technique involves the use of a secret value
- Second preimage resistant (**Weak collision resistant**)
  - it is impossible to find an alternative message with the same hash value as a given message.
  - This prevents forgery when an encrypted hash code is used.
- Collision resistant (**Strong collision resistance**)

Properties : Fixed length



- Arbitrary-length message to fixed-length digest

Hash Functions Family

- **MD (Message Digest)**
  - Designed by Ron Rivest
  - Family: MD2, MD4, MD5
- **SHA (Secure Hash Algorithm)**
  - Designed by NIST
  - Family: SHA-0, SHA-1, and SHA-2
    - SHA-2: SHA-224, SHA-256, SHA-384, SHA-512
    - SHA-3: New standard in competition
- **RIPEMD (Race Integrity Primitive Evaluation Message Digest)**
  - Developed by Katholieke University Leuven Team
  - Family : RIPEMD-128, RIPEMD-160, RIPEMD-256, RIPEMD-320

Distribution of Public Keys

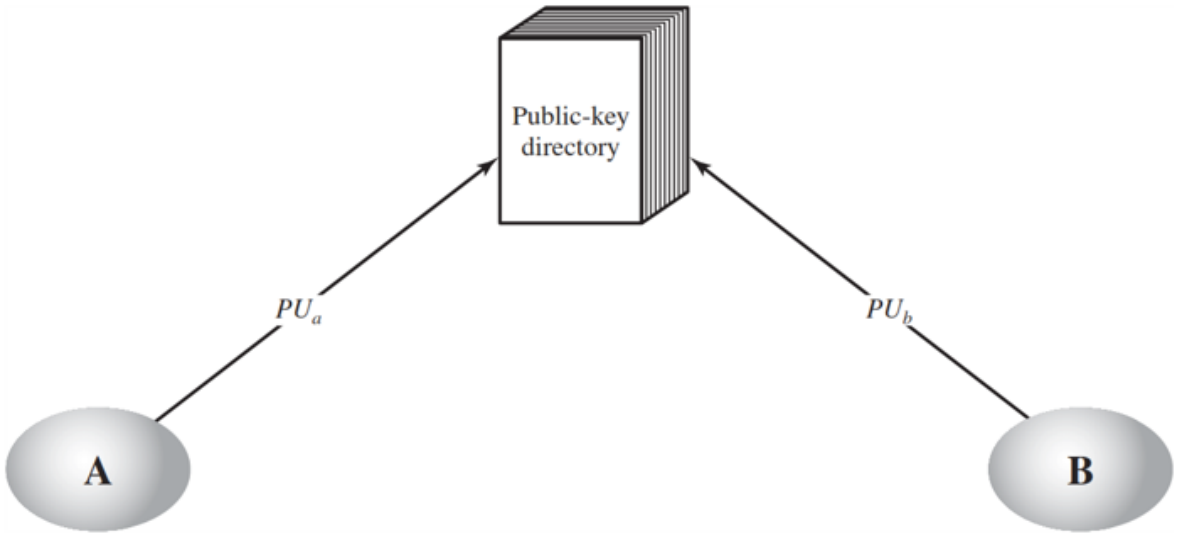
1. Public announcement
2. Publicly available directory
3. Public-key authority
4. Public-key certificates

1. Public Announcement



- Some user could pretend to be user A and send a public key to another participant or broadcast such a public key.
- Until such time as user A discovers the forgery and alerts other participants, the forger is able to read all encrypted messages intended for A and can use the forged keys for authentication

2. Publicly Available Directory

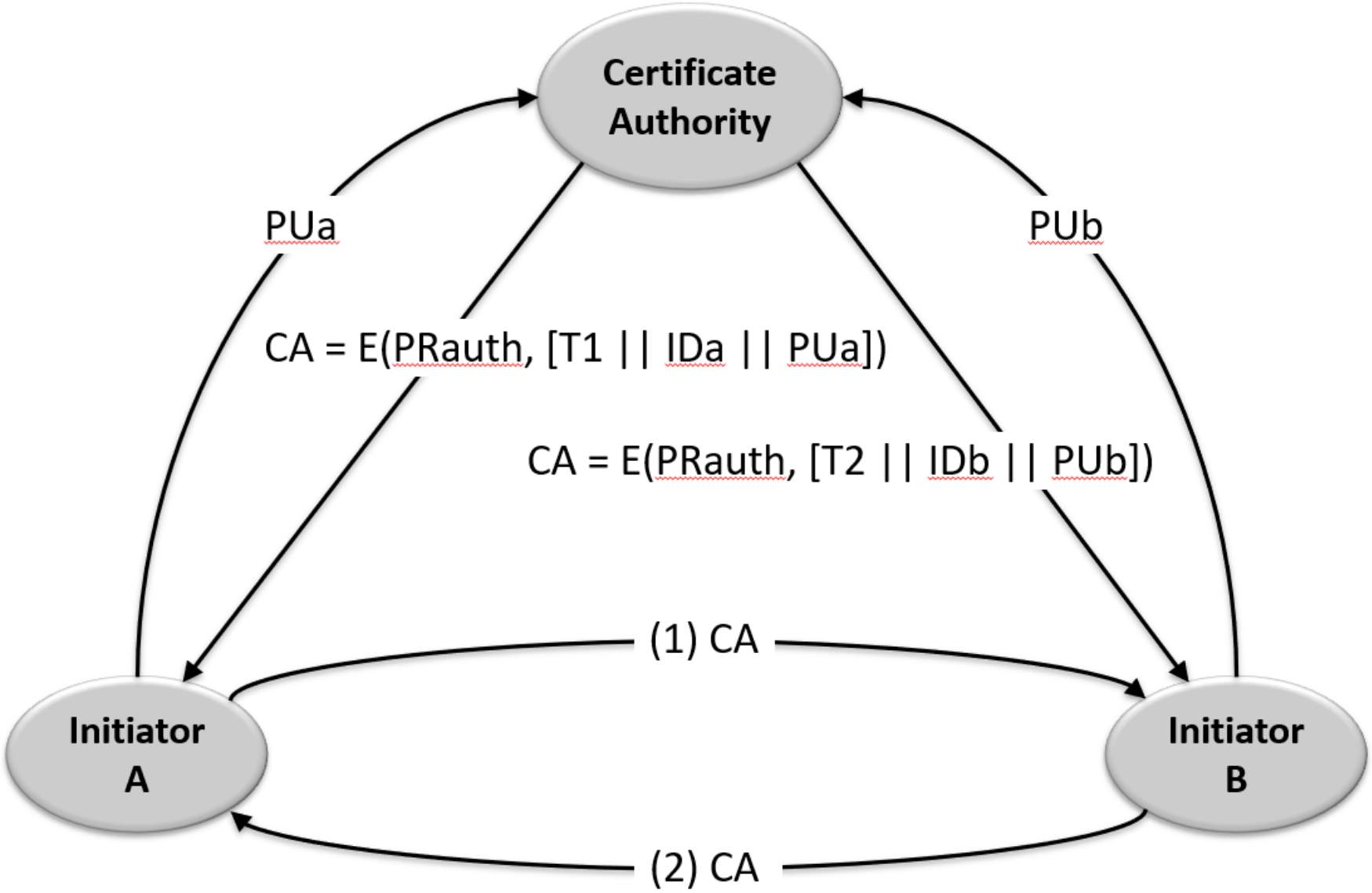


1. The **authority** maintains a directory with a **{name, public key}** entry for each participant.
2. Each participant registers a public key with the directory authority.
3. A participant may replace the existing key with a new one at any time.
4. Participants could also access the directory electronically. For this purpose, secure, authenticated communication from the authority to the participant is mandatory.

4. Public-Key Certificates

- Any participant can read a **certificate** to determine the name and public key of the certificate's owner.
- Any participant can verify that the **certificate** originated from the **certificate authority** and is not counterfeit.
- Only the **certificate authority** can create and update certificates.
- Any participant can verify the currency of the certificate.

4. Public-Key Certificates – Cont…



4. Public-Key Certificates – Cont…

- Each participant applies to the **certificate authority**, supplying a public key and requesting a certificate.
- For participant A, the authority provides a certificate of the form

$$CA = E(PR_{auth}, [T \,||\, ID_a \,||\, PU_a])$$

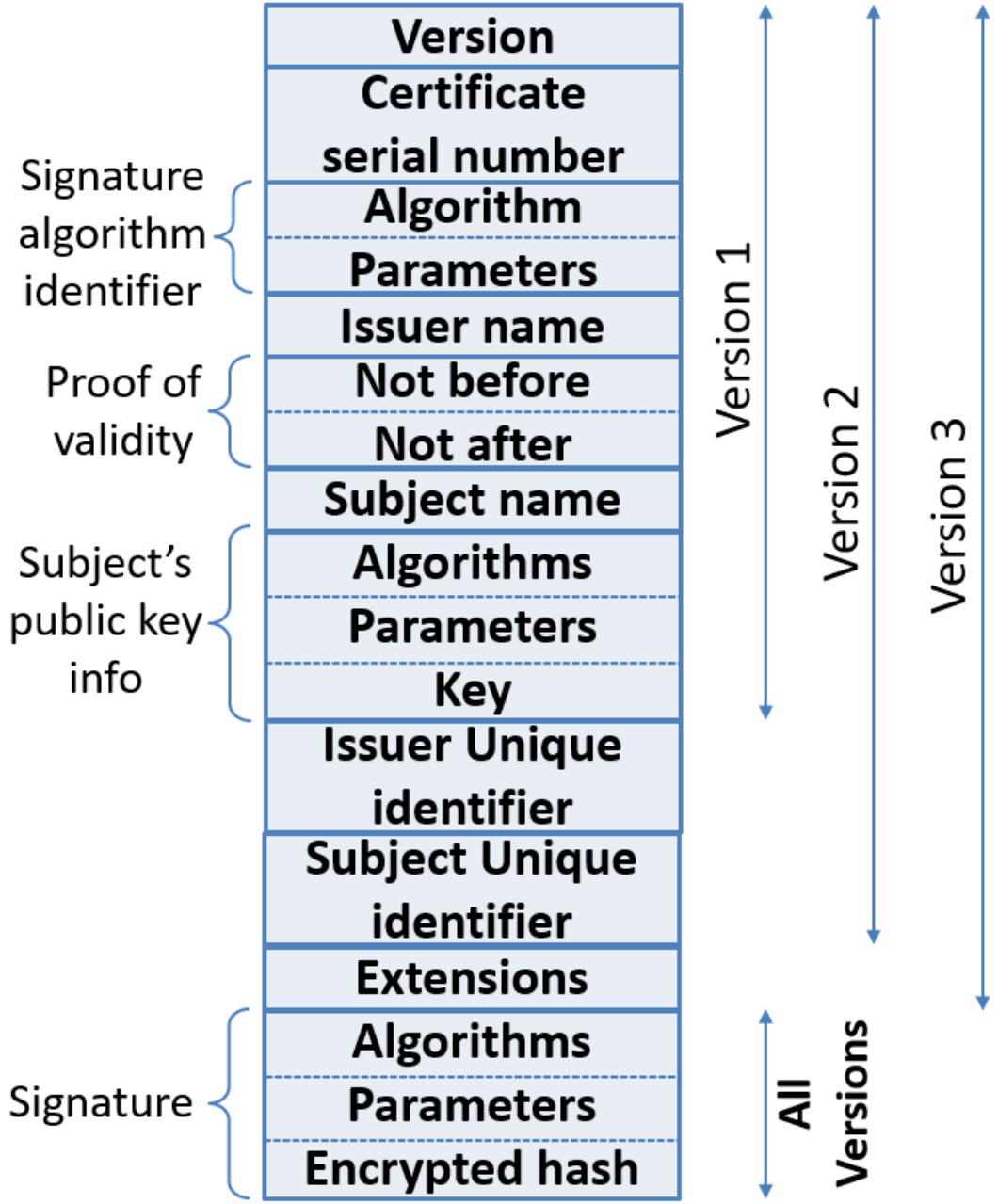
- A may then pass this certificate on to any other participant, who reads and verifies the certificate as follows:

$$D(PU_{auth}, CA)$$
$$= D(PU_{auth}, E(PR_{auth}, [T \,||\, ID_a \,||\, PU_a]))$$
$$= (T \,||\, ID_a \,||\, PU_a)$$

X.509 Certificates

- **X.509** defines the format for public-key certificates. used in a variety of applications.
- **X.509** defines a framework for the provision of authentication services by the X.500 directory to its users.
- The directory may serve as a repository of public-key certificates.
- Each certificate contains the **public key of a user** and is **signed with the private key** of a trusted certification authority.

X.509 Formats



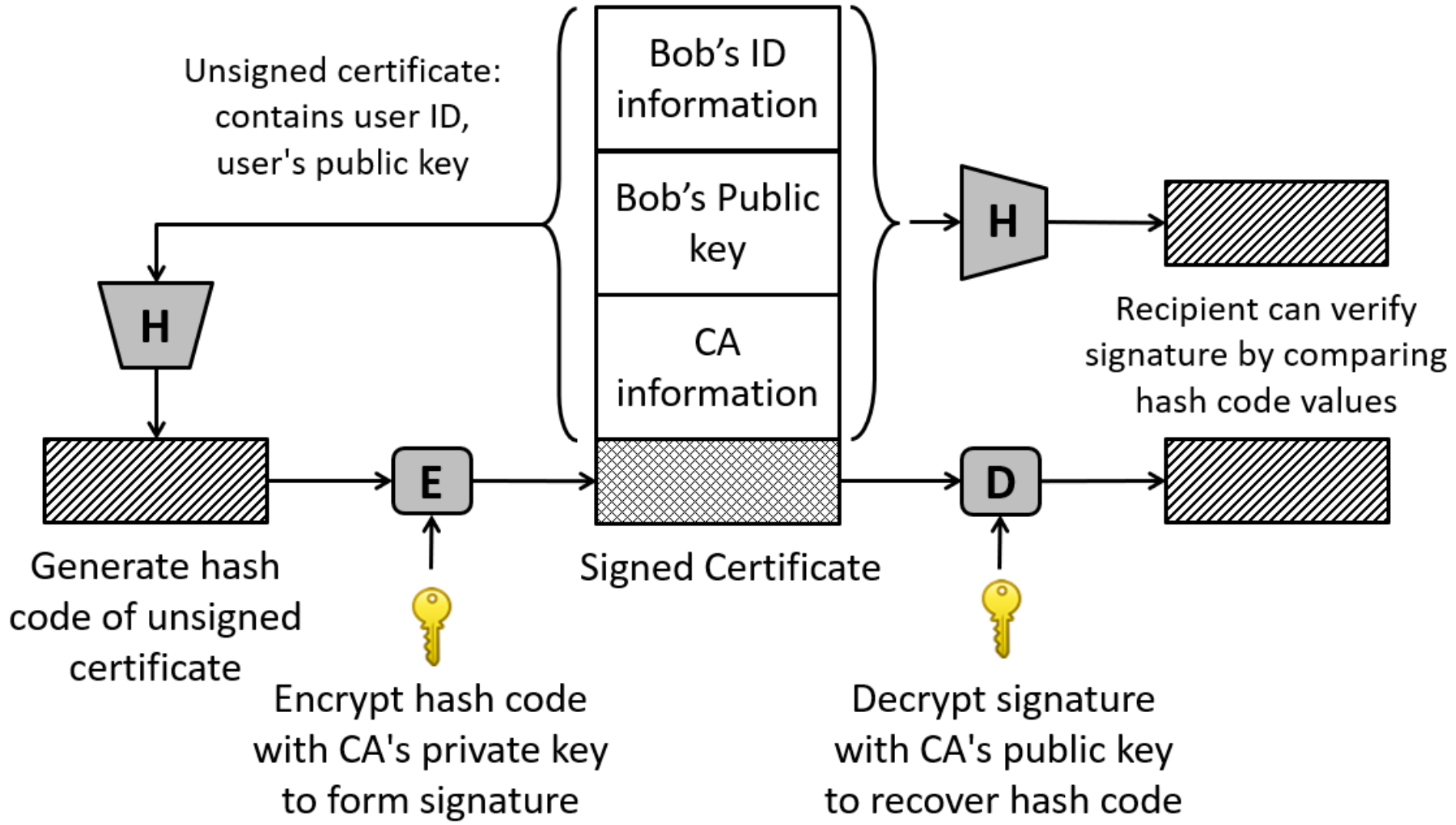
X.509 Format – Cont…

- **Version:** Differentiates among successive versions of the certificate format; the default is version 1.
- **Serial number:** An integer value unique within the issuing CA that is unambiguously associated with this certificate.
- **Signature algorithm identifier:** The algorithm used to sign the certificate together with any associated parameters.
- **Issuer name:** X.500 name of the CA that created and signed this certificate.
- **Period of validity:** Consists of two dates: the first and last on which the certificate is valid.
- **Subject name:** The name of the user to whom this certificate refers.

X.509 Format – Cont…

- **Subject's public-key information:** The public key of the subject, plus an identifier of the algorithm for which this key is to be used, together with any associated parameters.
- **Issuer unique identifier:** An optional-bit string field used to identify uniquely the issuing CA in the event the X.500 name has been reused for different entities.
- **Subject unique identifier:** An optional-bit string field used to identify uniquely the subject in the event the X.500 name has been reused for different entities.
- **Extensions:** A set of one or more extension fields.

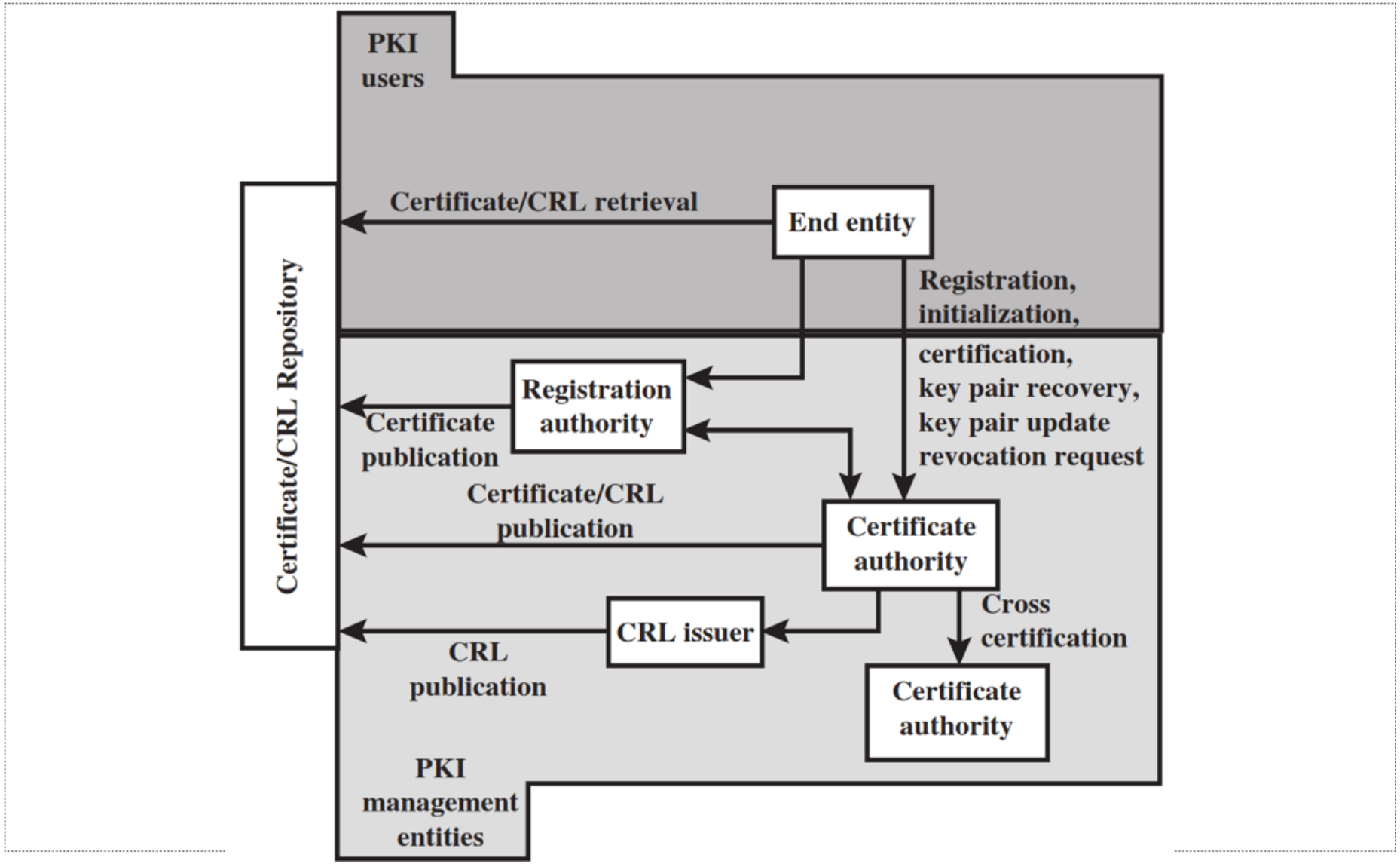
Public-Key Certificate Use



Public key Infrastructure (PKI)

- A **public-key infrastructure (PKI)** is defined as the set of hardware, software, people, policies, and procedures needed to create, manage, store, distribute, and revoke digital certificates based on asymmetric cryptography.
- The principal objective for developing a PKI is to enable secure, convenient, and efficient acquisition of **public keys**.

Public key Infrastructure (PKI)



Public key Infrastructure (PKI) – Cont…

- **End entity:** A generic term used to denote end users, devices (e.g., servers, routers), or any other entity that can be identified in the subject field of a public-key certificate.
- **Certification authority (CA):** The issuer of certificates and (usually) certificate revocation lists (CRLs).
- **Registration authority (RA):** An optional component that can assume a number of administrative functions from the CA.
- **CRL issuer:** An optional component that a CA can delegate to publish CRLs.
- **Repository:** A generic term used to denote any method for storing certificates and CRLs so that they can be retrieved by end entities.